

# Database Systems: Foundations, Architecture, Performance and Privacy

Sergio Peignier and Nicolas Parisot

## Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Foundational Concepts: Information Systems, Databases, and DBMS</b> | <b>4</b>  |
| 1.1      | Definitions . . . . .                                                  | 4         |
| 1.2      | Core Objectives of a DBMS . . . . .                                    | 5         |
| 1.2.1    | Basic Data Operations and Persistence . . . . .                        | 5         |
| 1.2.2    | The Transactional Consistency Spectrum: ACID vs. BASE . . . . .        | 5         |
| 1.2.3    | Scalability and Networks: The CAP Theorem . . . . .                    | 7         |
| 1.2.4    | Scalability and Performance in the Big Data Era . . . . .              | 7         |
| 1.2.5    | Multi-User Security and Accessibility . . . . .                        | 8         |
| 1.3      | Data Independence and the Three Levels of Abstraction . . . . .        | 8         |
| 1.4      | Taxonomy and Industry Landscape of DBMS Engines . . . . .              | 8         |
| 1.5      | The Database Lifecycle (DBLC) . . . . .                                | 8         |
| <b>2</b> | <b>Conceptual Data Modeling: The Entity-Association (EA) Framework</b> | <b>10</b> |
| 2.1      | Objectives of Conceptual Modeling . . . . .                            | 10        |
| 2.2      | Core Components of the Entity-Association Model . . . . .              | 10        |
| 2.2.1    | Entity Types and Dominated Entities . . . . .                          | 10        |
| 2.2.2    | Associations . . . . .                                                 | 10        |
| 2.2.3    | Properties, Attributes, and Domains . . . . .                          | 11        |
| 2.3      | Key Constraints and Cardinality Rules . . . . .                        | 11        |
| 2.3.1    | Primary, Candidate, and Composite Keys . . . . .                       | 11        |
| 2.3.2    | Cardinality Notation Conventions . . . . .                             | 12        |
| 2.4      | Systemic Modeling Methodology . . . . .                                | 12        |
| 2.5      | Evaluation of the Model Paradigm . . . . .                             | 12        |
| 2.6      | Exercise: Comprehensive Modeling Synthesis Problem . . . . .           | 13        |
| <b>3</b> | <b>Foundations of Relational Database Management Systems</b>           | <b>14</b> |
| 3.1      | Definitions and Formalization . . . . .                                | 14        |
| 3.2      | Anomalies and Relational Design . . . . .                              | 15        |
| 3.3      | Functional Dependencies . . . . .                                      | 15        |
| 3.3.1    | Mathematical Definition . . . . .                                      | 15        |
| 3.3.2    | Armstrong's Axioms and Secondary Rules . . . . .                       | 16        |
| 3.3.3    | Typologies of Functional Dependencies . . . . .                        | 16        |
| 3.4      | Closure Computations and Minimal Covers . . . . .                      | 17        |
| 3.4.1    | Attribute Closure ( $X^+$ ) . . . . .                                  | 17        |
| 3.4.2    | Transitive Closure ( $F^+$ ) and Minimal Cover ( $MIN(F)$ ) . . . . .  | 17        |
| 3.5      | Normal Forms . . . . .                                                 | 18        |

|          |                                                                                                                  |           |
|----------|------------------------------------------------------------------------------------------------------------------|-----------|
| 3.5.1    | First Normal Form (1NF)                                                                                          | 18        |
| 3.5.2    | Second Normal Form (2NF)                                                                                         | 19        |
| 3.5.3    | Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF)                                                        | 19        |
| 3.5.4    | Decomposition Framework via Heath's Theorem                                                                      | 19        |
| 3.5.5    | Join Dependency and Non-Loss Decompositions                                                                      | 20        |
| 3.5.6    | Comprehensive Synthesis Decomposition Problem                                                                    | 20        |
| 3.6      | Relational Algebra                                                                                               | 21        |
| 3.6.1    | Operands and Fundamental Operators                                                                               | 21        |
| 3.6.2    | Derived and Auxiliary Relational Operators                                                                       | 22        |
| <b>4</b> | <b>Practice Exercises</b>                                                                                        | <b>24</b> |
| <b>5</b> | <b>NoSQL Database Management Systems</b>                                                                         | <b>26</b> |
| 5.1      | Major NoSQL Paradigms                                                                                            | 26        |
| 5.1.1    | Key-Value Databases                                                                                              | 26        |
| 5.1.2    | Document-Oriented Databases                                                                                      | 26        |
| 5.1.3    | Graph-Oriented Databases                                                                                         | 26        |
| 5.1.4    | Column-Oriented Databases                                                                                        | 26        |
| 5.2      | Choosing a DBMS                                                                                                  | 27        |
| 5.2.1    | Advantages of NoSQL Systems                                                                                      | 27        |
| 5.2.2    | Disadvantages of NoSQL Systems                                                                                   | 27        |
| 5.2.3    | Choosing Between NoSQL and SQL                                                                                   | 27        |
| 5.2.4    | Choosing among NoSQL Systems                                                                                     | 28        |
| <b>6</b> | <b>Privacy: a Philosophical Perspective</b>                                                                      | <b>29</b> |
| 6.1      | Definitions                                                                                                      | 29        |
| 6.2      | Importance of Privacy                                                                                            | 29        |
| 6.3      | Privacy as an Essential Shield against Extrinsic Harm                                                            | 30        |
| 6.4      | The Panoptic Destruction of Free Will and Virtue                                                                 | 30        |
| 6.5      | The Interception of the Path toward Truth and the Good                                                           | 31        |
| 6.6      | The Danger of Dataist Paradigm                                                                                   | 32        |
| <b>7</b> | <b>Privacy Under Siege: Social Engineering, Legal Tracking, Corporate Aggregation and Malicious Exploitation</b> | <b>34</b> |
| 7.1      | Social Engineering: Hacking the Human Subsystem                                                                  | 34        |
| 7.1.1    | Mass Phishing and Deceptive Domain Cloning                                                                       | 34        |
| 7.1.2    | Smishing and Vishing                                                                                             | 34        |
| 7.2      | Legal Tracking: Breaking Privacy under Legality                                                                  | 34        |
| 7.2.1    | Abusive Application Permissions                                                                                  | 35        |
| 7.2.2    | Web Tracking: From Cookies to Device Fingerprinting                                                              | 35        |
| 7.2.3    | Foundational Defensive Frameworks for Web Tracking                                                               | 35        |
| 7.3      | Infrastructure vulnerabilities exploitation                                                                      | 36        |
| 7.3.1    | SQL Injection (SQLi)                                                                                             | 36        |
| 7.3.2    | Brute Force Attacks and Credential Security                                                                      | 36        |
| 7.4      | Corporate Aggregation, Breaches, and Linkage Attacks                                                             | 37        |
| 7.4.1    | Linkage Attacks                                                                                                  | 37        |
| 7.4.2    | Data Aggregation and Spear Phishing                                                                              | 37        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>8</b> | <b>Foundations of Data Bases Privacy</b>               | <b>39</b> |
| 8.1      | The Illusion of Anonymity in Data Ecosystems . . . . . | 39        |
| 8.2      | Taxonomy of Attributes in Data Privacy . . . . .       | 40        |
| <b>9</b> | <b>The Defender Mindset</b>                            | <b>42</b> |
| 9.1      | Preventing SQLi . . . . .                              | 42        |
| 9.2      | Mitigating Data Linkage . . . . .                      | 42        |
| 9.2.1    | $k$ -Anonymity . . . . .                               | 42        |
| 9.2.2    | $l$ -Diversity and $t$ -Closeness (Bonus) . . . . .    | 43        |
| 9.2.3    | Differential Privacy $\epsilon$ -DP (Bonus) . . . . .  | 44        |
| 9.3      | Trade-off Between Privacy and Utility . . . . .        | 45        |

# 1 Foundational Concepts: Information Systems, Databases, and DBMS

## 1.1 Definitions

To analyze how software applications process real-world facts, we must decouple the core theoretical definitions that govern information frameworks.

### Theoretical Foundations

- **System:** A defined collective of interacting or interdependent elements operating according to structured, deterministic principles. For example, the biological human body itself is a system, where interconnected organs interact to produce physical signs.
- **Data:** A raw collection of observations, measurements, facts, or primitive symbols. Data represents potential messages but lacks contextual framing. For example, "140" could be a data instance. Without context, it lacks meaning.
- **Information:** A structured, organized set of data that provides context, enabling the extraction of actionable knowledge. For example when the data instance "140" is contextualized, it becomes information : *"The patient's systolic blood pressure is 140 mmHg, indicating Stage 1 hypertension."*

### Infrastructure and Software Layers

- **Information System (IS):** The complete ecosystem of combined resources (including personnel, hardware, software networks, and operational methodologies) engineered to systematically **collect, store, analyze, query, and disseminate** information across an enterprise. For example, an Information System (IS) is the entire hospital ecosystem: the computers, the doctors typing notes, the internal Wi-Fi networks, the administrative staff, and the workflows that coordinate patient care.
- **Database (DB):** A structured, cohesive ensemble of digitally stored data modeled to represent real-world objects, processes, or entities. This data layer serves as the persistent foundation for computational applications. For example, a DB is the specific digital table holding operational records, such as active patient names, assigned room numbers, and current medication schedules.
- **Database Management System (DBMS):** The specialized system software engine engineered to govern the creation, maintenance, security, and transaction management of a database. For instance, a DBMS is the underlying software engine (e.g., MySQL) ensuring that two different receptionists cannot accidentally book the exact same operating room for the same hour.

### Analytical Storage Architectures

- **Data Lake:** A massive, permanent repository that stores raw data in its native format (structured, semi-structured, and unstructured) until it is needed, employing a *Schema-on-Read* approach. For example, a Data Lake is a massive storage repository where the hospital dumps raw, unfiltered files for long-term retention: high-resolution X-ray images, audio recordings of emergency calls, and raw log files from continuous heart monitors.

- **Data Warehouse:** A centralized, highly structured repository optimized for analytical querying and reporting, containing only cleaned, transformed, and modeled data. For instance, a Data Warehouse is a refined analytical database where a data engineer loads cleaned financial and clinical metrics, allowing the hospital executive to open a clean dashboard and view the monthly state of the cardiology department.
- **Data Swamp:** A degraded Data Lake that has become chaotic, untagged, and unmanageable due to a lack of metadata, proper governance, and data lifecycle management. For instance, a Data Swamp occurs if the hospital uploads millions of medical images into the Data Lake without names, timestamps, or metadata tags, rendering the files completely unidentifiable and useless.

## Deployment Models

- **On-Premise (Local):** A deployment model where an organization hosts, maintains, and retains physical ownership of its hardware infrastructure within its own local facility. For instance, if a hospital buys cables, cools, and locks up physical servers in the basement of its own building, it is operating **On-Premise**.
- **Cloud (Platform):** A deployment model where computational power and storage are provisioned over the internet as an on-demand, subscription-based service managed by a third-party provider (e.g., AWS, Azure, GCP). For example, if a hospital dismantles its basement servers and rents compute power and storage from Amazon Web Services (AWS) via a monthly subscription, it has migrated to the **Cloud**.

## 1.2 Core Objectives of a DBMS

A modern enterprise DBMS acts as an intermediary between the physical storage hardware and application code, executing several foundational and advanced sub-objectives:

### 1.2.1 Basic Data Operations and Persistence

At a baseline level, a DBMS must ensure, to some extent, long-term data persistence, enforce a reliable structural model layout, and expose high-throughput mechanisms to query and modify existing database elements.

### 1.2.2 The Transactional Consistency Spectrum: ACID vs. BASE

**The ACID Transactional Paradigm [11]** To guarantee data validity and operational sanity across high-concurrency environments, a DBMS must natively enforce the standard **ACID** properties for every transaction:

- **Atomicity:** Multi-statement transactions are executed as a single, indivisible unit of work. If any single query fails, the entire transaction is aborted and rolled back. Example: Consider a bank transfer where the DBMS must execute a two-step sequence: first, deduct \$100 from Alice's account, and second, credit \$100 to Bob's account. If the database successfully subtracts the money from Alice but the server loses power before it can credit Bob, Atomicity forces the DBMS to completely undo the first step, rolling back Alice's balance to its original state so that the money does not vanish.

- **Consistency:** Transactions can only move the database from one valid state to another, preserving all declared structural constraints and business rules. Example: Suppose a bank database

enforces a structural integrity constraint dictating that no account balance can drop below \$0. If Alice has \$50 in her account and attempts to transfer \$100 to Bob, the DBMS evaluates the system rules, determines that the resulting negative balance would violate this predefined constraint, and immediately aborts the transaction to preserve the structural sanity of the schema.

- **Isolation:** Concurrency control mechanisms isolate concurrently executing transactions from one another, preventing dirty or unrepeatable reads. Example: Imagine Alice transfers \$100 to Bob at the exact same millisecond that an automated auditing script reads both of their account balances. Isolation ensures that the auditing script cannot view an intermediate, half-baked state where the money has left Alice's account but has not yet arrived in Bob's; the script is forced to read the balances either entirely before the transfer begins or entirely after it commits.

- **Durability:** Once a transaction is successfully committed, its updates are written to non-volatile storage and will persist even through system crashes or power failures. Example: The moment a bank transfer completes and the system displays a "Success" screen to the user, the DBMS immediately flushes the transaction logs from temporary RAM onto a persistent, non-volatile solid-state drive. If a catastrophic power failure instantly shuts down the data center a split-second later, Durability guarantees that Bob's updated balance remains permanently written and will not be lost when the servers reboot.

**The BASE Properties [15]** To facilitate horizontal scalability and partition tolerance, the so-called "NoSQL database management systems" often abandon the strict **ACID** framework in favor of the more flexible **BASE** paradigm:

- **Basic Availability:** Because data is highly distributed and heavily replicated across multiple nodes, the system ensures a high level of availability even in the presence of localized node failures or under high traffic conditions. Example: While a team is working on a project proposal, one of the primary cloud database nodes hosting their document suffers a sudden hardware failure. Thanks to Basic Availability, the document editor does not crash, instead, the system instantly routes their active editing sessions to live replica nodes in a different data center, ensuring the team can keep working.

- **Soft State:** The data state can fluctuate or change over time without explicit user intervention due to background replication mechanisms. Furthermore, data states could potentially expire unless actively maintained. Example: As team members type paragraphs simultaneously, the internal state of the document across the cloud infrastructure continuously fluctuates and drifts without any explicit saving action from the users. The system's data state remains dynamic and fluid as background synchronization protocols constantly exchange operational differences between user browsers and regional servers, quietly changing the underlying data values behind the scenes to reflect the evolving document.

- **Eventual Consistency:** The system offers no absolute guarantee of immediate consistency at individual transaction levels. However, because data is systematically replicated across multiple nodes, the system eventually converges toward a synchronized, consistent state over time. Example: If a writer in Tokyo deletes a paragraph, a collaborator in Paris might still see that paragraph on their screen for a brief fraction of a second due to cross-continental network latency. Rather than freezing the document to force an instant global update, the system prioritizes smooth typing, guaranteeing through Eventual Consistency that within a few moments the synchronization messages will resolve, the conflicts will merge, and both screens will converge onto the exact same final version of the document.

### 1.2.3 Scalability and Networks: The CAP Theorem

The structural divergence between ACID and BASE is governed by absolute physical and network constraints formalized by Eric Brewer in the **CAP Theorem** [5]. The theorem states that any distributed database system operating over a network can simultaneously guarantee only **two out of the following three properties**:

- **Consistency**: Every node request returns either the most recent write or an error. (Note that this is distinct from the “Consistency” definition used within traditional ACID transactional paradigms). Example: While writing in a shared document, the network partition cuts off the connection between continents, a collaborator in Madrid types a sentence while a collaborator in Lima tries to read the document. A system prioritizing Consistency realizes the Lima server cannot verify if Madrid has made changes, so it immediately locks the document, turns the screen to read-only, or displays an error message; it deliberately sacrifices availability, choosing to block the user rather than risking showing them out-of-date or conflicting text.

- **Availability**: Every node request receives a non-error response, even if other nodes in the network have failed. However, the system does not guarantee that the response contains the most recent updates. Example: When the connection drops between the regional servers, a system prioritizing Availability ensures that both the user in Lima and the user in Madrid can keep typing smoothly without a single error or interruption. The Lima server accepts updates locally, and the Madrid server accepts updates locally, deliberately sacrificing immediate consistency and allowing the two versions of the document to temporarily drift apart so that neither user experiences downtime.

- **Partition Tolerance**: The system continues to operate properly in the presence of network partitions, such as when connectivity drops or messages between nodes are delayed or lost. Example: During a major undersea cable malfunction, the primary cloud data center loses communication with its regional backup data center across the ocean. Because the document editing platform is built with Partition Tolerance, the entire application does not experience a fatal crash; instead, both data centers continue running independently, gracefully handling their local users’ edits and storing them safely until the network partition heals and the servers can synchronize again.

Because physical hardware networks will inevitably experience occasional delays or partitions, **Partition Tolerance (P) is non-negotiable** for distributed software engineering. Therefore, a modern system architect must make an explicit choice between two configurations:

- **CP Systems (Consistency + Partition Tolerance)**: The database chooses absolute ACID consistency. If a network partition prevents nodes from synchronizing, the system rejects incoming writes and returns errors to guarantee data validity, sacrificing Availability.

- **AP Systems (Availability + Partition Tolerance)**: The database chooses BASE availability. Under a network partition, all nodes continue accepting writes locally to keep the app online, accepting the fact that data will temporarily desynchronize across regions until the partition heals.

### 1.2.4 Scalability and Performance in the Big Data Era

Modern DBMS engines are optimized to ingest and process massive volumes of data scaling into the terabyte and petabyte ranges using efficient indexing structures (such as B-Trees or LSM-Trees). Depending on their position on the CAP/BASE spectrum, they must handle high-throughput workloads, processing thousands or millions of concurrent read/write queries per hour.

### 1.2.5 Multi-User Security and Accessibility

A DBMS must secure sensitive records using robust authentication, authorization, and cryptographic access boundaries. It must accommodate diverse target audiences by providing specialized user interfaces (such as raw CLI tools, administrative GUIs, or programmatic APIs) while managing high-concurrency environments.

## 1.3 Data Independence and the Three Levels of Abstraction

A principal objective of database design is achieving **data independence**, which decouples application code from the underlying physical storage details. This is achieved via the ANSI-SPARC<sup>1</sup> three-level architecture model:

- **The External Level (User Views):** The top level, describing how individual end-users or application routes experience and interact with partial subsets of the data.
- **The Conceptual Level (Logical Model):** The central blueprint managed by database designers and administrators. It defines the universal logical schema, entities, relationships, functional dependencies, and security constraints.
- **The Internal Level (Physical Storage Layout):** The lowest level, specifying how data is physically arranged on disk, mapping file blocks, index allocations, data compression, and hashing algorithms.

**The Mechanics of Independence:** By structuring a DBMS around these distinct boundaries, the system guarantees two essential architectural principles:

- **Physical Data Independence:** The internal disk file structure or indexing mechanism can be completely overhauled without breaking the conceptual schemas or altering application-tier code.
- **Logical Data Independence:** The conceptual logical schema can be refactored or expanded (e.g., adding attributes or splitting tables) without disrupting the external application views or rewriting current client queries.

## 1.4 Taxonomy and Industry Landscape of DBMS Engines

The modern database ecosystem features over 300 active, production-grade engines tracked across community indexes like *DB-Engines*. These systems are categorized into two primary architectural families:

- **Relational DBMS (RDBMS):** Highly structured systems anchored by strict relational schemas and queried using standard SQL (e.g., Oracle, MySQL, SQLite, PostgreSQL, Microsoft SQL Server).
- **NoSQL Systems:** Diverse non-relational storage engines optimized for flexible schemas, horizontal scaling, and specialized access patterns: (*Key-Value Stores* (e.g., Redis), *Document Stores* (e.g., MongoDB), *Wide-Column / Columnar Stores* (e.g., Cassandra), *Graph Databases* (e.g., Neo4j), *Time Series and Object-Oriented Engines*

## 1.5 The Database Lifecycle (DBLC)

A database implementation follows a structured lifecycle to ensure alignment with system demands:

- 1) Specification Definition
- 2) Conceptual Modeling and DBMS Selection
- 3) Implementation

---

<sup>1</sup>American National Standards Institute, Standard Planning and Requirements Committee

#### 4) Production Use and Maintenance

## 2 Conceptual Data Modeling: The Entity-Association (EA) Framework

### 2.1 Objectives of Conceptual Modeling

Before committing an information layout to a physical database configuration or a specific database management system (DBMS), engineers construct a symbolic and reduced abstract layout of the target enterprise ecosystem [21, 6]. This process produces a **Conceptual Representation** that maps business requirements independently of the target storage implementation or hardware hosting architecture.

The two main modeling paradigms used in database engineering are:

- **Object-Oriented Models:** Layout frameworks mapping abstractions directly to programming structures (e.g., Unified Modeling Language (UML) Class Diagrams).
- **Entity-Association (EA) Models:** Graph-based structural techniques that map entities, keys, and operational dependencies.

Historically, multiple data schools of thought formalized specific graphic variants of the EA model. Among the most prominent frameworks are **Chen's notation**, **Bachman diagrams**, **Teorey's methodology**, and **Merise**. The Merise framework serves as the core teaching standard for this lecture.

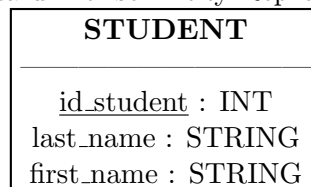
### 2.2 Core Components of the Entity-Association Model

An Entity-Association (EA) layout provides a definitive graphical overview mapping three distinct structural elements: entity types, semantic associations, and descriptive attributes.

#### 2.2.1 Entity Types and Dominated Entities

An **Entity Type** (informally called an Entity) provides a distinct conceptual layout representing a real-world object, actor, or transactional event. It possesses a clear, independent operational identity and its own unique set of properties.

Figure 1: Standard Merise Entity Representation Layout

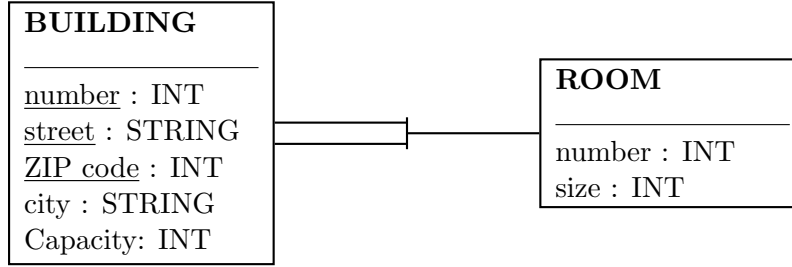


Conversely, when an object cannot exist standalone in the ecosystem, it is classified as a **Dependent Entity** (also referred to as a **Dominated** or **Weak Entity**). The existence of a dominated entity is conditioned entirely by the concurrent presence of at least one independent parent entity block. Figure 2 represents a dominated and dominating entity relationship.

#### 2.2.2 Associations

An **Association** represents a semantic link or conceptual relationship connecting separate entities. While the model mathematically allows  $n$ -ary relationships connecting more than two entities

Figure 2: Dominant (BUILDING) and dominated (ROOM) entities



( $n > 2$ ), keeping associations binary is strongly recommended to simplify structural parsing. The framework natively accommodates **reflexive associations**, where an entity establishes a relational loop linking back to itself.

Figure 3: Standard Binary Association Layout



### 2.2.3 Properties, Attributes, and Domains

To store data facts within this graph, attributes are mapped across both entity and association nodes:

- **Property:** An elementary piece of information that independently describes an aspect of an entity or association.
- **Attribute:** The formal implementation of a property embedded inside a model block.
- **Domain:** The explicit set of values an attribute can legally house, operating identically to data types in programming languages (e.g., `STRING`, `INTEGER`, `BOOLEAN`).

## 2.3 Key Constraints and Cardinality Rules

### 2.3.1 Primary, Candidate, and Composite Keys

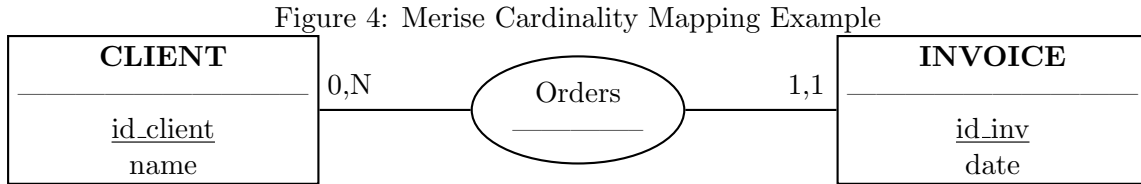
A **Key** within an entity is defined as the minimal subset of attributes that uniquely identifies every discrete row instance of that entity.

- **Candidate Keys:** The set of all unique attributes or combinations capable of acting as an entity identifier.
- **Primary Key:** The definitive identifier chosen by the system designer. In graphics layouts, the primary key is denoted by underlining its label. All other unselected candidates become **Secondary Keys**.
- **Composite Key:** An identifier generated by combining multiple attributes because no single property can guarantee uniqueness.

If an entity has no valid identifier, it is classified as a weak/dominated entity. A robust primary key should require no subsequent modifications and maintain minimal memory storage overhead.

### 2.3.2 Cardinality Notation Conventions

**Cardinality** defines the numeric range governing how entities interact across an association. It sets the absolute minimum and maximum number of times an individual entity instance can participate in a given relationship.



While different industry notations exist the Merise standard uses localized (*min, max*) tuple strings placed along the connecting relationship line:

- **Minimum Cardinality (0 or 1):** Specifies if the relationship is completely optional (0) or strictly mandatory (1).
- **Maximum Cardinality (1 or N):** Specifies if an entity instance can link to at most a single item (1) or an unrestricted collection of records (*N*).

## 2.4 Systemic Modeling Methodology

To build an enterprise-grade conceptual schema without architectural errors, engineers follow the following steps:

1. Map Entities → Assign Attributes
2. Assigned Attributes → Choose Keys
3. Entities and Keys → Establish Associations
4. Established Associations → Attach Association Attributes
5. Established Associations → Define Cardinalities
6. Model Completed → Validate Coherence

## 2.5 Evaluation of the Model Paradigm

**Advantages:** The EA paradigm provides a clear, high-level graphical layout that remains simple yet expressive. It enables non-technical stakeholders and system engineers to collaborate and quickly model database designs for standard production ecosystems.

**Limitations:** The model can become cluttered and hard to read when handling complex enterprise architectures. Additionally, the quality of a schema relies heavily on the experience of the designer, and the framework entirely lacks native operations or query mechanisms to manipulate data once recorded.

## 2.6 Exercise: Comprehensive Modeling Synthesis Problem

**System Specifications:** Design a clean Merise Entity-Relationship model matching the following production requirements for a television network infrastructure:

A TV series is uniquely identified by its name and is split into multiple seasons. Each season has a specific sequential identifier and an official release date. Each season contains a series of episodes. Every individual episode is identified by a sequential episode number (which restarts at 1 with each new season), holds a unique title, and must feature a minimum of two actors. An actor is tracked by a unique INSEE identification number, a last name, a first name, and a specific contract salary that changes dynamically on an episode-by-episode basis.

### 3 Foundations of Relational Database Management Systems

The Relational Database Management Systems (RDBMS) model was originally proposed by **E.F. Codd** [7], a contribution for which he later received the Turing Award in 1981. Although the first commercial relational systems began emerging in the late 1970s, it was not until the early 1980s that database engineers developed highly efficient query optimization engines capable of competing with legacy hierarchical and network database architectures.

#### 3.1 Definitions and Formalization

The relational framework is built entirely upon established mathematical disciplines:

- **Set Theory:** Data items are treated as members of distinct mathematical sets, where ordering is irrelevant and duplicates are strictly managed.

- **Mathematical Relations:** An  $n$ -ary relation over  $n$  sets is defined rigorously as a **subset of the Cartesian product** of those  $n$  sets.

Because data structures are modeled directly as mathematical relations, formal algebraic operations can be systematically applied to them. Crucially, due to the property of **closure**, the execution of these relational operations always outputs a new mathematical relation, enabling complex query nesting.

**Core Relational Definitions** To translate these mathematical concepts into software engineering abstractions, we define the practical building blocks of an RDBMS:

- **Relational Database:** A finite collection or set of mathematical relations. For example, a bookstore's entire storage engine is a Relational Database, meaning it is structured as a finite, mathematically sound collection of distinct but interconnected relations. Rather than piling all information into a single unstructured text file, the database keeps a set of specific relations (e.g., BOOKS, AUTHORS, ORDERS); where the mathematical relationships between these sets ensure that no order can exist without a valid book and no book can exist without a valid author.

- **Relation (Table):** An unordered, unique collection of rows on disk. For instance, the BOOKS Relation is an unordered, unique collection of records stored persistently on the server's disk drives. Because a mathematical relation is inherently an unsorted set, the database engine does not guarantee that books will be read or returned in the order they were inserted; furthermore, the system guarantees uniqueness by ensuring that no two identical rows representing the exact same physical print copy can coexist within this storage structure.

- **Column (Attribute):** A named property within a relation mapped to a specific data type or data constraints, formally known as a **domain**. For example: within the BOOKS relation, the price Column acts as a named property that defines a very specific characteristic of every book record. This attribute is strictly mapped to a specific data type and domain constraint, ensuring that the application can only write valid positive monetary amounts into this slot and preventing anyone from accidentally typing text like "Free" into the price field.

- **Row (Tuple):** A concrete instance of an entity record. Every row contains exactly one structural value corresponding to each declared attribute. When a particular value is missing or unknown, SQL frameworks use the special value token NULL. For example, a single Row in the BOOKS relation represents one concrete, real-world book instance, such as a tuple containing the values (9780131873254, 'An Introduction to Database Systems', 85.50, NULL). Every single declared attribute in the table gets exactly one structural value in this row, and because the publication date for this specific historical textbook edition was omitted during data entry, the database utilizes the special NULL token to safely signify that the date value is currently missing or unknown.

**Schema Definitions and Compatibility Constraints** An explicit distinction is made between the layout structure of a database and its active row contents:

**Schema:** The permanent structural description of the database relations, tables, types, and constraints. Formally, the schema of a specific relation is denoted as follows:

$$R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$$

where  $R$  is the unique name of the relation,  $A_i$  represents the explicit name of the  $i$ -th attribute,  $D_i$  denotes the underlying domain (the valid set of possible values) restricted to that attribute, and  $n$  represents the operational **degree** (or arity) of the relation.

**Schema Compatibility:** To execute key set-based operations (such as unions, intersections, and differences) between two distinct relations  $A$  and  $B$ , the relations must be explicitly declared **compatible** (or union-compatible). Compatibility requires meeting two rigid criteria:

- Relation  $A$  and relation  $B$  must share the exact same cardinality/degree ( $n$ ).
- For every index position  $i$ , the  $i$ -th domain of relation  $A$  must be identical to the  $i$ -th domain of relation  $B$  ( $\forall i, D_{A_i} = D_{B_i}$ ).

### 3.2 Anomalies and Relational Design

When mapping application requirements to a relational schema, engineers encounter multiple valid logical options. However, opting for an unadapted or unnormalized model introduces structural systemic flaws that create severe runtime anomalies:

- **Redundancy:** Instances are recorded multiple times on disk, bloating storage consumption.
- **Update Anomalies:** Altering a real-world attribute requires modifying thousands of rows individually, risking systemic desynchronization.
- **Insertion Anomalies:** The database cannot record certain facts because they are logically tied to other unrecorded entities.
- **Deletion Anomalies:** Purging a transactional record inadvertently deletes underlying static master domain facts.

To ground these issues, consider a non-normalized spreadsheet model for tracking food items:

FOOD\_PRODUCT(Name : *STR*, Address : *STR*, Product\_name : *STR*, Price : *FLOAT*)

If a business location moves, every record containing that location must be modified. If a location temporarily has no menu items, deleting those menu items completely removes the location's address history from the system. Relational **Normalization** resolves these operational vulnerabilities by verifying that schemas conform to strict mathematical boundaries called **Normal Forms** [19].

### 3.3 Functional Dependencies

Normal forms are formally constructed using the concept of **Functional Dependencies (FDs)** [19], which model the invariant constraints linking sets of attributes within a relational schema.

#### 3.3.1 Mathematical Definition

Let  $A$  and  $B$  represent two distinct subsets of attributes within a relation schema  $R$ . Let  $x$  and  $y$  represent two arbitrary rows (tuples) within a legal instance of  $R$ . Let  $x_A$  denote the explicit projection of values for attribute set  $A$  within row  $x$ .

**Definition:** If for every possible row pair  $x$  and  $y$ , equality on the values of attribute set  $A$  mathematically guarantees equality on the values of attribute set  $B$ :

$$x_A = y_A \implies x_B = y_B$$

Then we state that  $B$  is **functionally dependent on**  $A$ , or that  $A$  **structurally determines**  $B$ . This structural relationship is designated by the notation:

$$A \rightarrow B$$

For example, Let  $A$  be the set of attributes {STUDENT ID} and  $B$  be the set of attributes {NAME, LAST NAME}. If we inspect any two arbitrary rows  $x$  and  $y$  in the student registry table and find that they have the exact same value for {STUDENT ID} ( $x_A = y_A$ ), it is mathematically and logically impossible for them to have different {NAME, LAST NAME} values, meaning their name and last names must also be identical ( $x_B = y_B$ ). Because {STUDENT ID} uniquely and unambiguously identifies a specific student, it structurally determines their identity, allowing us to formally state that {NAME, LAST NAME} is functionally dependent on {STUDENT ID}, written notationally as:

$$\{\text{STUDENT ID}\} \rightarrow \{\text{NAME, LAST NAME}\}$$

Functional dependencies are derived strictly from the core rules of the domain semantics. They can **never** be validated by simply inspecting an instance table snapshot, as coincidental row values frequently mimic functional dependencies that do not exist in practice.

### 3.3.2 Armstrong's Axioms and Secondary Rules

Given an initial set of fundamental FDs, database designers can deduce the complete set of valid dependencies using three primary axioms [19]:

- **Reflexivity:** If  $B \subseteq A$ , then  $A \rightarrow B$ .
- **Augmentation:** If  $A \rightarrow B$ , then  $AC \rightarrow BC$  (where  $AC$  denotes  $A \cup C$ ).
- **Transitivity:** If  $A \rightarrow B$  and  $B \rightarrow C$ , then  $A \rightarrow C$ .

From these foundational axioms, three critical secondary inference rules are mathematically derived:

$$\begin{aligned} \textbf{Union:} \quad & \text{If } A \rightarrow B \text{ and } A \rightarrow C, \text{ then } A \rightarrow BC \\ \textbf{Pseudo-Transitivity:} \quad & \text{If } A \rightarrow B \text{ and } DB \rightarrow C, \text{ then } DA \rightarrow C \\ \textbf{Decomposition:} \quad & \text{If } A \rightarrow B \text{ and } C \subseteq B, \text{ then } A \rightarrow C \end{aligned}$$

### 3.3.3 Typologies of Functional Dependencies

- **Trivial FD:** A dependency derived purely from set containment rules ( $A \rightarrow B$  is trivial if  $B \subseteq A$ ).
- **Simple FD:** A dependency that targets a single attribute on the right side ( $A \rightarrow B$  is simple if  $|B| = 1$ ).
- **Direct FD:** A dependency that does not rely on intermediate attributes (there is no set  $C$  such that  $A \rightarrow C$  and  $C \rightarrow B$ ).

- **Full FD:** A dependency where the left-hand determinant set cannot be reduced further ( $\forall C \subset A, C \not\rightarrow B$ ). If a subset  $C$  can determine  $B$ , the dependency is classified as a **Partial FD**.
- **Irreducible / Minimal FD:** A dependency that is concurrently **Simple** and **Full**.

### 3.4 Closure Computations and Minimal Covers

#### 3.4.1 Attribute Closure ( $X^+$ )

Let  $X$  represent a subset of attributes belonging to relation  $R$ , and let  $F$  define the comprehensive family of active FDs. The **attribute closure** [19], denoted as  $X^+$ , is the complete set of all attributes within  $R$  that can be logically inferred from  $X$  using the complete set of FDs via Armstrong's axioms. Consequently, any attribute set  $Y$  is included within  $X^+$  if and only if  $X \rightarrow Y$  is valid.

#### Algorithmic Computation of $X^+$ :

1. **Initialization:** Set  $X^+ = X$ .
2. **Iteration Loop:** Scan the family of FDs for any dependency  $A \rightarrow B$  such that  $A \subseteq X^+$ . If found, update the closure set:

$$X^+ \leftarrow X^+ \cup B$$

3. **Termination:** Repeat step 2 until the attribute members of  $X^+$  undergo no further additions.

**Analytical Exercise:** Consider the family of dependencies  $F = \{A \rightarrow D; AB \rightarrow E; BI \rightarrow E; CD \rightarrow I; E \rightarrow C\}$ .

- To find the closure  $(AE)^+$ : Initial state is  $AE$ . The dependency  $A \rightarrow D$  applies since  $A \subseteq AE$ , bringing in  $D$  (state:  $AED$ ). Next,  $E \rightarrow C$  applies, bringing in  $C$  (state:  $AEDC$ ). Finally,  $CD \rightarrow I$  applies since  $CD \subseteq AEDC$ , bringing in  $I$ . Thus,  $(AE)^+ = AEDCI$ .
- To find the closure  $(BE)^+$ : Initial state is  $BE$ . The rule  $E \rightarrow C$  applies, expanding to  $BEC$ . No other left-hand determinant matches this set. Thus,  $(BE)^+ = BEC$ .

#### 3.4.2 Transitive Closure ( $F^+$ ) and Minimal Cover ( $MIN(F)$ )

The **transitive closure of an FD set** [19], denoted  $F^+$ , represents the total theoretical maximum space of all non-trivial, irreducible functional dependencies that can be derived from the core set  $F$ . Two distinct sets of functional dependencies are mathematically declared **equivalent** if and only if they generate an identical transitive closure ( $F_1^+ = F_2^+$ ).

The **Minimal Cover** [19], or  $MIN(F)$ , represents a reduced canonical equivalent of  $F$  stripped of all redundant attributes and dependencies. Any functional dependency family can be mapped to at least one minimal cover using a three-step optimization algorithm.

#### Minimal Cover Generation Algorithm Workflow:

- **Step 1: Simplify Right-Hand Attributes.** Decompose all non-simple dependencies into a set of independent simple dependencies. Any rule of the format  $X \rightarrow A_1A_2 \dots A_n$  is split into  $n$  separate expressions:  $X \rightarrow A_i$ .

- **Step 2: Eliminate Extraneous Left-Hand Attributes.** For each multi-attribute determinant (e.g.,  $Y \rightarrow Z$ ), test if a sub-component can be pruned. If an attribute  $B \in Y$  can be removed such that  $(Y \setminus \{B\}) \rightarrow Z$  remains inferable from the rest of the system,  $B$  is structurally redundant and is removed.
- **Step 3: Eliminate Redundant Dependencies.** Evaluate every single remaining dependency one by one. Temporarily remove the dependency under test,  $X \rightarrow A$ , from the set. If  $A$  can still be resolved within the attribute closure  $X^+$  using only the remaining dependencies, the tested dependency is redundant and is purged.

**Algorithmic Execution Example:** Let  $F = \{A \rightarrow B; ABCD \rightarrow E; EF \rightarrow GH; ACDF \rightarrow EG\}$ .

- **Execution of Step 1:** Split compound right sides to isolate simple attributes:

$$F = \{A \rightarrow B; ABCD \rightarrow E; EF \rightarrow G; EF \rightarrow H; ACDF \rightarrow E; ACDF \rightarrow G\}$$

- **Execution of Step 2:** Evaluate left sides for redundant elements. In  $ABCD \rightarrow E$ , since we have  $A \rightarrow B$ , the attribute  $B$  can be dropped, converting the rule to  $ACD \rightarrow E$ . In  $ACDF \rightarrow E$ , the presence of  $ACD \rightarrow E$  renders  $F$  extraneous, reducing the expression to  $ACD \rightarrow E$ . The system updates to:

$$F = \{A \rightarrow B; ACD \rightarrow E; EF \rightarrow G; EF \rightarrow H; ACD \rightarrow E; ACDF \rightarrow G\}$$

- **Execution of Step 3:** Clean out duplicate rules and transitively derivable links. Pruning one instance of the duplicate  $ACD \rightarrow E$  leaves a single copy. Next, evaluate  $ACDF \rightarrow G$ : since  $ACD \rightarrow E$  and  $EF \rightarrow G$ , we can trace  $ACDF \rightarrow EF \rightarrow G$ , making  $ACDF \rightarrow G$  redundant. Pruning it reveals the final minimal cover:

$$\text{MIN}(F) = \{A \rightarrow B; EF \rightarrow G; EF \rightarrow H; ACD \rightarrow E\}$$

### 3.5 Normal Forms

#### 3.5.1 First Normal Form (1NF)

A relation schema  $R$  satisfies **First Normal Form** [19] if and only if every attribute contained within its schema houses strictly **atomic values**, and the relation contains a clearly defined primary key to prevent duplicate tuples.

**Case Study: The Multi-Valued Array Flaw.** Consider a medical phone directory schema:

Table 1: Non-1NF Initial Phone Layout

| Name             | Phone_Numbers | Address         |
|------------------|---------------|-----------------|
| Homer Simpson    | 06.01, 06.66  | Street A, # 65  |
| Ned Flanders     | 06.02         | Street B, # 66  |
| Montgomery Burns | 06.03, 06.67  | Street H, # 666 |

This structure explicitly fails 1NF because **Phone\_Numbers** maps non-atomic strings containing multiple distinct entries. This creates major operational flaws: inserting a row without a phone number is blocked, updates require parsing strings, and deleting a phone number risks erasing the entire row. Converting this schema to 1NF requires decomposing the values into distinct rows to restore atomicity:

Table 2: Normalized 1NF Tabular Layout

| Name (PK)        | Phone_Number (PK) | Address         |
|------------------|-------------------|-----------------|
| Homer Simpson    | 06.01             | Street A, # 65  |
| Homer Simpson    | 06.66             | Street A, # 65  |
| Ned Flanders     | 06.02             | Street B, # 66  |
| Montgomery Burns | 06.03             | Street H, # 666 |
| Montgomery Burns | 06.67             | Street H, # 666 |

### 3.5.2 Second Normal Form (2NF)

A relation schema  $R$  satisfies **Second Normal Form** [19] if it is in 1NF and every non-key attribute depends fully on the complete primary key. Formally, given primary key  $K$ , for any attribute  $A \notin K$ , there cannot exist a strict proper subset  $C \subset K$  such that  $C \rightarrow A$ .

**Analytical Diagnostic Case:** Consider an enterprise allocation schema:

Employee(idEmp, idService, name, salary, nameService)

where the primary key is compound: [idEmp, idService]. The system states the following underlying FDs:

idEmp  $\rightarrow$  name, salary    and    idService  $\rightarrow$  nameService

This model explicitly **fails 2NF** because the attributes *name* and *salary* depend on a proper subset of the key (*idEmp*), rather than the combined key. This creates structural redundancy that can only be resolved by decomposing the schema.

### 3.5.3 Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF)

A relation schema  $R$  satisfies **Third Normal Form** [19] if it is in 2NF and exhibits no transitive dependencies among non-key attributes. Formally, for every non-trivial functional dependency  $X \rightarrow A$ , at least one of the following conditions must hold:

$X$  is a superkey of  $R$     or     $A$  is a prime attribute (part of a candidate key)

The stricter **Boyce-Codd Normal Form (BCNF)** removes the prime attribute exception completely, enforcing that for every single active non-trivial dependency  $X \rightarrow A$ , the determinant  $X$  **must** be a superkey<sup>2</sup>.

### 3.5.4 Decomposition Framework via Heath's Theorem

To transform an unnormalized schema into its proper normal forms without losing underlying dependency semantics or existing tuple facts, systems apply formal decomposition.

---

<sup>2</sup>A superkey is simply a set of one or more attributes within a database table that **uniquely identifies each** record in that table

### 3.5.5 Join Dependency and Non-Loss Decompositions

Let  $R_1, R_2, \dots, R_n$  represent a decomposition layout splitting a universal relation  $R$ . The schema satisfies a non-loss join dependency if reconstructing the pieces via natural joins<sup>3</sup> matches the original relation exactly, without generating spurious rows:

$$\bowtie_{i=1}^n \Pi_{R_i}(R) = R$$

To execute this split, database engines leverage **Heath's Theorem**: if relation  $R(A, B, C)$  exhibits the functional dependency  $B \rightarrow C$ , it can be split without data loss into two separate relation modules:

$$R_1(A, B) \quad \text{and} \quad R_2(B, C)$$

### 3.5.6 Comprehensive Synthesis Decomposition Problem

Map the following universal schema into a clean 3NF layout:

Universal Relation:  $R(A, B, C, D, E, F, G, H, I, J, K)$

Minimal Cover:  $F = \{AB \rightarrow CE; A \rightarrow F; F \rightarrow DGH; D \rightarrow IJK\}$

#### Algorithmic Resolution Steps:

1. **Find the Key and Establish its Boundaries:** Compute the attribute closure of candidate keys (e.g.,  $AB, A, F, D$ ). For instance  $(F)^+$  yields  $F \rightarrow DGH$  and  $D \rightarrow IJK$ , which does not encompass all attributes, hence  $F$  is not the key. The combination  $(AB)^+$  yields  $AB \rightarrow CE$ ,  $A \rightarrow F \rightarrow DGH$ , and  $D \rightarrow IJK$ , which encompasses every attribute in the schema. Thus, **AB** is the primary key.
2. **Isolate Partial Key Dependencies (2NF Corrections):** The rule  $A \rightarrow F$  exposes a partial key dependency since  $A \subset AB$ . Applying Heath's Theorem splits out  $R_A(A, F)$ , leaving  $R_{\text{main}}(A, B, C, D, E, G, H, I, J, K)$ .
3. **Isolate Transitive Dependencies (3NF Corrections):** Inside  $R_A(A, F)$ , the dependency  $F \rightarrow DGH$  introduces transitivity. This requires creating  $R_F(F, D, G, H)$  and updating the parent layer to  $R_A(A, F)$ .
4. **Resolve Terminal Transitive Dependencies:** Within the branch containing  $D$ , the rule  $D \rightarrow IJK$  indicates another transitive dependency. This creates the terminal relation  $R_D(D, I, J, K)$ , leaving the updated parent as  $R_F(F, D, G, H)$ .

The final non-loss, 3NF schema structure is:

$$\mathbf{R_1(A, B, C, E)}, \mathbf{R_2(A, F)}, \mathbf{R_3(F, D, G, H)}, \mathbf{R_4(D, I, J, K)}$$

---

<sup>3</sup>The different Relational Algebra Operators are defined in the following section

## 3.6 Relational Algebra

### 3.6.1 Operands and Fundamental Operators

Relational algebra is a formal, procedural query language that takes one or two relations as inputs and produces a new relation as a result. Relational algebra operates on **constant relations** or **variable relations** with a fixed, predefined arity. While its Fundamental Operators; from which all other operations can be derived, are the Set Union ( $\cup$ ), the Set Difference ( $-$ ), the Cartesian Product ( $\times$ ), the Projection ( $\pi$ ), and the Selection ( $\sigma$ ). To illustrate each operator, consider two sample relations  $R$  and  $S$  representing enrolled students in two distinct courses:

| Relation $R$ (Database Course) |              |               | Relation $S$ (AI Course) |              |               |
|--------------------------------|--------------|---------------|--------------------------|--------------|---------------|
| ID ( $A$ )                     | Name ( $B$ ) | Grade ( $C$ ) | ID ( $A$ )               | Name ( $B$ ) | Grade ( $C$ ) |
| 101                            | Alice        | A             | 102                      | Bob          | B             |
| 102                            | Bob          | B             | 104                      | David        | A             |
| 103                            | Charlie      | A             |                          |              |               |

**Set Union** ( $R \cup S$ ) Combines all distinct tuples from relation  $R$  and relation  $S$ .  $R$  and  $S$  must be **union-compatible** (same arity and compatible attribute domains). Formally, the set union can be defined as  $R \cup S = \{t \mid t \in R \vee t \in S\}$ . The equivalent SQL command is: `SELECT * FROM R UNION SELECT * FROM S;`

| Example Output ( $R \cup S$ ) = |              |               |
|---------------------------------|--------------|---------------|
| ID ( $A$ )                      | Name ( $B$ ) | Grade ( $C$ ) |
| 101                             | Alice        | A             |
| 102                             | Bob          | B             |
| 103                             | Charlie      | A             |
| 102                             | Bob          | B             |
| 104                             | David        | A             |

**Set Difference** ( $R - S$ ) Returns all tuples that are present in relation  $R$  but not present in relation  $S$ . Requires union compatibility. Formally, the set difference can be defined as  $R - S = \{t \mid t \in R \wedge t \notin S\}$ . The equivalent SQL command is: `SELECT * FROM R EXCEPT -- SELECT * FROM S;.`

| Example Output ( $R - S$ ) = |              |               |
|------------------------------|--------------|---------------|
| ID ( $A$ )                   | Name ( $B$ ) | Grade ( $C$ ) |
| 101                          | Alice        | A             |
| 103                          | Charlie      | A             |

**Cartesian Product** ( $R \times S$ ) Combines every tuple from relation  $R$  (arity  $k_1$ ) with every tuple from relation  $S$  (arity  $k_2$ ). The resulting relation has arity  $k_1 + k_2$ . Formally, the cartesian product can be defined as  $R \times S = \{t_1 \cdot t_2 \mid t_1 \in R \wedge t_2 \in S\}$ . The equivalent SQL command is: `SELECT * FROM R CROSS JOIN S;` or alternatively: `SELECT * FROM R, S;.` In pour previous example, ( $R \times S$ ): Generates  $3 \times 2 = 6$  combined tuples with schema ( $R.A, R.B, R.C, S.A, S.B, S.C$ ).

**Example Output**  $(R \times S) =$ 

| ID (R.A) | Name (R.B) | Grade (R.C) | ID (S.A) | Name (S.B) | Grade (S.C) |
|----------|------------|-------------|----------|------------|-------------|
| 101      | Alice      | A           | 102      | Bob        | B           |
| 102      | Bob        | B           | 102      | Bob        | B           |
| 103      | Charlie    | A           | 102      | Bob        | B           |
| 101      | Alice      | A           | 104      | David      | A           |
| 102      | Bob        | B           | 104      | David      | A           |
| 103      | Charlie    | A           | 104      | David      | A           |

**Projection**  $(\pi_{B_1, \dots, B_m}(R))$  Selects specified attributes (columns)  $B_1, \dots, B_m$  from relation  $R$  and eliminates duplicate rows from the result. More formally, let  $B \subseteq \text{Attributes}(R)$ , the the projection is simply  $\pi_B(R) = \{t[B] \mid t \in R\}$ . The equivalent SQL command is **SELECT DISTINCT Name, Grade FROM R;**

**Example Output**  $(\pi_{\text{Name, Grade}}(R)) =$ 

| Name (B) | Grade (C) |
|----------|-----------|
| Alice    | A         |
| Bob      | B         |
| Charlie  | A         |

**Selection**  $(\sigma_F(R))$  Filters rows from relation  $R$  based on a logical predicate formula  $F$ .  $F$  includes attribute names, constants, comparison operators ( $=, \neq, <, \leq, >, \geq$ ), and logical operators ( $\wedge, \vee, \neg$ ). More formally, the selection is defined as  $\sigma_F(R) = \{t \in R \mid F(t) = \text{true}\}$ . The equivalent SQL command is **SELECT \* FROM R WHERE Grade = 'A';**

**Example output**  $(\sigma_{\text{Grade}='A'}(R)) =$ 

| ID (A) | Name (B) | Grade (C) |
|--------|----------|-----------|
| 101    | Alice    | A         |
| 103    | Charlie  | A         |

### 3.6.2 Derived and Auxiliary Relational Operators

**Set Intersection**  $(R \cap S)$  Yields all tuples present in both  $R$  and  $S$ . This operator can be derived from Fundamental Operators as:  $R \cap S = R \cup S - (R - S) - (S - R)$  or alternatively  $R \cap S = R - (R - S)$ . The equivalent SQL command is **SELECT \* FROM R INTERSECT SELECT \* FROM S;**

**Example output**  $(R \cap S) =$ 

| ID (A) | Name (B) | Grade (C) |
|--------|----------|-----------|
| 102    | Bob      | B         |

**Theta Join**  $(R \bowtie_{\theta} S)$  Combines tuples from  $R$  and  $S$  that satisfy a specified predicate condition  $\theta$ . This operator can be derived from Fundamental Operators as:  $R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$ . The corresponding SQL command is **SELECT \* FROM R JOIN S ON R.Grade = S.Grade;—**

| Example Output ( $R \bowtie_{R.Grade=S.Grade} S$ ) = |            |             |          |            |             |
|------------------------------------------------------|------------|-------------|----------|------------|-------------|
| ID (R.A)                                             | Name (R.B) | Grade (R.C) | ID (S.A) | Name (S.B) | Grade (S.C) |
| 102                                                  | Bob        | B           | 102      | Bob        | B           |
| 101                                                  | Alice      | A           | 104      | David      | A           |
| 103                                                  | Charlie    | A           | 104      | David      | A           |

**Natural Join** ( $R \bowtie S$ ) Joins relations  $R$  and  $S$  on all attributes having identical names in both schemas, keeping only matching tuples and projecting out duplicate common columns. The corresponding SQL command is simply `SELECT * FROM R NATURAL JOIN S;`. Notice that in SQL you can also find other JOIN operators including INNER JOIN, OUTER JOIN, LEFT JOIN. The procedure to compute this operator using basic operators is:

1. Compute  $R \times S$ .
2. Apply selection  $\sigma$  to match all common attribute pairs ( $R.A_i = S.A_i$ ).
3. Apply projection  $\pi$  to drop duplicated attribute columns.

| Example Output ( $R \bowtie S$ ) = |      |       |
|------------------------------------|------|-------|
| ID                                 | Name | Grade |
| 102                                | Bob  | B     |

**Division Operator** ( $R \div S$ ) Finds tuples in  $R$  (arity  $r$ ) that are paired with **every** tuple in  $S$  (arity  $s$ , where  $r > s$ ). Useful for "for all" queries. The derivation of this operator from Fundamental Operators is  $R \div S = \pi_{1,\dots,r-s}(R) - \pi_{1,\dots,r-s}((\pi_{1,\dots,r-s}(R) \times S) - R)$ . This operator is less commonly used in SQL, and its equivalent SQL operator is: `SELECT R1.A FROM R R1 WHERE NOT EXISTS (SELECT S.B FROM S EXCEPT SELECT R2.B FROM R R2 WHERE R2.A = R1.A);`

## Core Definitions

- **Relation:** A set of  $k$ -tuples (where  $k$  is the **arity** of the relation).
- **Operands:** **constant relations** or **variable relations** with a fixed, predefined arity.
- **Fundamental Operators:**
  1. Set Union ( $\cup$ )
  2. Set Difference ( $-$ )
  3. Cartesian Product ( $\times$ )
  4. Projection ( $\pi$ )
  5. Selection ( $\sigma$ )

- **Derived Operators:**

1. **Set Intersection ( $\cap$ ):**

$$R \cap S = R - (R - S)$$

2. **Theta Join ( $\bowtie_{\theta}$ ):**

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

3. **Natural Join ( $\bowtie$ ):**

$$R \bowtie S = \pi_{\text{attributes}}(\sigma_{\text{common matches}}(R \times S))$$

4. **Division ( $\div$ ):**

$$R \div S = \pi_{1,\dots,r-s}(R) - \pi_{1,\dots,r-s}((\pi_{1,\dots,r-s}(R) \times S) - R)$$

## 4 Practice Exercises

### Exercise 1: TV Series Relational Database Model

*Schema Assumption:*  $Actor(\underline{actor\_id}, name), Series(\underline{series\_id}, title), Episode(\underline{ep\_id}, \underline{series\_id}, \underline{season\_num}, ep\_num), Cast(\underline{actor\_id}, \underline{ep\_id}, pay\_amount)$

1. **Actors who played in the first episode of each season of "Game of Thrones":**

$$\pi_{name}(Actor \bowtie Cast \bowtie \sigma_{ep\_num=1}(Episode \bowtie \sigma_{title='Game\ of\ Thrones'}(Series)))$$

2. **Actors who played in the first season of "Game of Thrones":**

$$\pi_{name}(Actor \bowtie Cast \bowtie \sigma_{season\_num=1}(Episode \bowtie \sigma_{title='Game\ of\ Thrones'}(Series)))$$

3. **Actors of S01E01 of "Game of Thrones" paid less than 10 dollars:**

$$\pi_{name}(Actor \bowtie \sigma_{pay\_amount < 10}(Cast) \bowtie \sigma_{season\_num=1 \wedge ep\_num=1}(Episode \bowtie \sigma_{title='Game\ of\ Thrones'}(Series)))$$

4. **Actors who played in both "Game of Thrones" and "The Mentalist":**

$$\pi_{actor\_id, name}(Actor \bowtie Cast \bowtie Episode \bowtie \sigma_{title='Game\ of\ Thrones'}(Series))$$

$\cap$

$$\pi_{actor\_id,name}(Actor \bowtie Cast \bowtie Episode \bowtie \sigma_{title='The\ Mentalist'}(Series))$$

## Exercise 2: Dinner Relational Database Model

*Schema Assumption: Pairing(dish, wine), Person(person\_id, name), Friendship(person\_id, friend\_id), Enmity(person\_id, enemy\_id), Likes(person\_id, dish)*

1. **Wines that can be provided with a "Cassoulet":**

$$\pi_{wine}(\sigma_{dish='Cassoulet'}(Pairing))$$

2. **Enemies of "Bart Simpson":**

$$\pi_{enemy\_id}(Enmity \bowtie \sigma_{name='Bart\ Simpson'}(Person))$$

3. **Enemies of the friends of "Bart Simpson":**

$$\pi_{Enmity.enemy\_id}(Enmity \bowtie_{person\_id=friend\_id}(\pi_{friend\_id}(Friendship \bowtie \sigma_{name='Bart\ Simpson'}(Person))))$$

4. **Enemies of people who enjoy eating "Cassoulet":**

$$\pi_{enemy\_id}(Enmity \bowtie \sigma_{dish='Cassoulet'}(Likes))$$

## 5 NoSQL Database Management Systems

The term **NoSQL** stands for “**Not Only SQL**”. It serves as an umbrella term encompassing a new generation of non-relational Database Management Systems (DBMS).

A key defining element of most NoSQL architectures is that they function as **distributed databases**. A distributed database is an architecture where data is systematically replicated and stored across a network of distinct physical locations, rather than residing on a single, centralized server.

### 5.1 Major NoSQL Paradigms

NoSQL solutions are broadly categorized into four primary design paradigms, each optimized for distinct operational tasks.

#### 5.1.1 Key-Value Databases

Key-value systems (e.g., [9]) store data as a collection of values indexed explicitly by unique keys. These systems can store both structured and unstructured data seamlessly by treating values simply as opaque arrays of bytes, meaning the database management system completely disregards the internal content of the payload during ingestion. This architecture provides maximum read and write performance, scaling capacity, simplicity, and flexibility. However, a key limitation is that they do not natively support complex query filters or multi-attribute searches, restricting operations primarily to direct key lookups. Prominent examples of this model include Amazon DynamoDB, Apache Cassandra, and Project Voldemort.

#### 5.1.2 Document-Oriented Databases

Document stores [17] share conceptual roots with key-value architectures but introduce data visibility: They structure their data payloads using standard semi-structured formats such as JSON (JavaScript Object Notation) or XML. Unlike key-value stores, the core difference here is that the database management system can parse, read, index, and natively query the individual fields contained directly inside the document values. This capability enables support for advanced semantic queries as well as partial document mutations. Common examples of document-oriented databases include Apache CouchDB and MongoDB.

#### 5.1.3 Graph-Oriented Databases

In this paradigm, the database is mathematically modeled as a graph structure [16] consisting of entities represented as **nodes**, which are explicitly interconnected via **edges** (relationships). In this paradigm, both nodes and edges can hold arbitrary key-value attributes. This structure allows deep queries to be executed dynamically across interconnected node and edge networks. While offering extreme modeling flexibility for highly interconnected networks, they can be complex to implement, and their performance and scaling characteristics vary significantly depending on overall graph density and traversal depth. Prominent examples of graph databases include Neo4j, Hyperbase-DB, and InfoGrid.

#### 5.1.4 Column-Oriented Databases

Column-oriented databases (e.g., [20]) share a visual similarity with traditional relational tables since they utilize table structures and often support SQL-like declarative queries. However, their

underlying disk storage mechanics are completely inverted:

Instead of writing a table to disk row by row, the data layout is segmented and stored by columns. Column-oriented databases are highly efficient for analytical workloads executing aggregations or operations across millions of values within a specific column. On the contrary, they tend to be inefficient for transactional operations attempting to read or write many different columns within a single individual row simultaneously.

Table 3: Physical Data Layout Comparison

| Logical Conceptual Table |      |        | Row-Oriented Layout | Column-Oriented Layout |
|--------------------------|------|--------|---------------------|------------------------|
| ID                       | Name | Salary |                     |                        |
| 001                      | Gogo | 10     | 001:Gogo,10         | Gogo:001,Didi:002      |
| 002                      | Didi | 9      | 002:Didi,9          | 10:001,9:002           |

## 5.2 Choosing a DBMS

### 5.2.1 Advantages of NoSQL Systems

NoSQL environments offer several operational advantages for modern data challenges:

- **No Predefined Data Model Schema:** NoSQL systems support flexible storage, making them ideal for handling heterogeneous, semi-structured, and completely unstructured datasets. The underlying data storage mechanisms are simple, leading to high-performance throughput with exceptionally fast read and write operations.
- **Horizontal Scaling:** As the storage volume and request traffic grow, additional processing and storage nodes can easily be added to the distributed network without requiring modifications to the application tier.
- **Open-Source Ecosystem:** Most prevalent NoSQL database engines are distributed as open-source software, reducing initial licensing overhead.

### 5.2.2 Disadvantages of NoSQL Systems

Conversely, engineering trade-offs include:

- **Non-Standardized Query Languages:** Unlike relational systems that rely globally on SQL, every NoSQL database management system uses its own proprietary query language and API syntax.
- **Lack of ACID Compliance:** There is a significantly weaker control over transaction Consistency, Isolation, and Durability, which can introduce edge cases during high-concurrency writes.
- **Unenforced Integrity Constraints:** Data integrity (such as domain restrictions, type validations, or handling null values) is often not natively handled by the database tier. Consequently, external validation layers or application-side functions must be implemented to ensure data sanity.

### 5.2.3 Choosing Between NoSQL and SQL

When choosing between SQL and NoSQL engines, engineers must systematically analyze the workload profile across several critical vectors: the **total database storage volume** (encompassing current data footprints and projected growth rates), the required direction and elasticity of **system scalability** (whether scaling vertically via hardware capacity or horizontally across distributed nodes), and the aggregate **throughput of read/write operations** executed per unit of time.

Furthermore, the architecture must optimize for the **specific types of operations** that dominate the workload—distinguishing between heavy analytical aggregations and rapid transactional point-queries—while robustly supporting the expected level of **multi-user concurrency** and satisfying the absolute enterprise mandates regarding **data integrity** and immediate consistency levels.

#### 5.2.4 Choosing among NoSQL Systems

The following architectural matrix establishes a clear baseline comparison between the four NoSQL paradigms and traditional relational database systems across five core metrics, graded using **H** (High), **M** (Medium), **L** (Low), or **Var** (Variable):

Table 4: Comprehensive DBMS Choice Matrix

| Metric               | Key-Value | Document | Graph    | Columnar | Relational |
|----------------------|-----------|----------|----------|----------|------------|
| <b>Performance</b>   | H         | H        | Variable | H        | Variable   |
| <b>Scalability</b>   | H         | H        | Variable | H        | Variable   |
| <b>Flexibility</b>   | H         | H        | H        | M        | L          |
| <b>Complexity</b>    | L         | L        | H        | L        | M          |
| <b>Functionality</b> | L         | M        | H        | M        | H          |

## 6 Privacy: a Philosophical Perspective

Before analyzing technical and legal perspectives it is necessary to understand that anonymity raises deep philosophical questions that address human freedom, truth and the notion of humanity itself. To establish a logically philosophical analysis, we must first delineate the foundational concepts underpinning our analysis. These definitions are presented through both Classical and Modern lens to illustrate their deep ontological implications.

### 6.1 Definitions

**Freedom and Free Will** A synthesis of the classical definitions of freedom could be formalized as the inherent human faculty to govern oneself, guided by reason, toward a higher good, while the modern view sees it as being free from external coercion <sup>4</sup>.

**Good** Similarly, synthesizing the classical and modern definitions of good, this concept could be defined as the objective perfection or plenitude of being toward which human nature tends through the autonomous cultivation of virtue <sup>5</sup>.

**Truth** In the Classical Perspective, Truth is primarily understood as the correspondence of the mind with objective reality [8, 1]. Its apprehension is a dynamic, lifelong exploration that inherently requires dialectic, trial, error, and intellectual exploration. The alignment of human intellect with objective reality, the discovery of which is a developmental process requiring independent experimentation, hypothesis testing, and the freedom to err.

**Privacy and Absolute Surveillance** **Privacy** can be defined as the existential and informational space reserved exclusively for the individual, shielded from the gaze, judgment, or interference of the public, the collective, or the State. While **Absolute Surveillance (Loss of Privacy)** would be the state of total exposure wherein an individual’s internal reflections, external behaviors, and digital exhaust are continuously visible to, and evaluated by, external authorities or the collective consciousness.

### 6.2 Importance of Privacy

Having established our conceptual taxonomy, we now formalize the core philosophical arguments into deductive syllogisms.

---

<sup>4</sup>According to the classical Perspective (e.g., Plato, Aristotle, Aquinas) Freedom is not the mere absence of external constraints. For Plato, it is the dominion of reason over the passions and ignorance [4]. For Aquinas, while free will is the faculty of choice, while true freedom is teleological: the capacity of the will, guided by reason, to choose the appropriate means to achieve the human ultimate end [1]. According to the Modern Perspective (enlightenment), Freedom is predominantly defined as negative liberty, as the absence of external obstacles, coercion, or interference by other agents or the State [3].

<sup>5</sup>According to the Classical Perspective, the concept of "Good" is ontological: In Plato’s metaphysics, the Form of the Good is the supreme reality, the source of all truth, intelligibility, and existence [4]. For Aristotle, the good of any entity is the realization of its proper function or purpose. In the case of man, the good is human flourishing achieved by living a life in accordance with virtue and reason [2]. Aquinas builds on this, defining the good as that which is desirable because it actualizes a being’s potential perfection [1]. According to the Modern Perspective derived from enlightenment, the good is often relativized, reduced to subjective utility, individual preference maximization, or legal/social consensus.

### 6.3 Privacy as an Essential Shield against Extrinsic Harm

Privacy acts as an essential shield that protects us from outside harm because without it, external actors can easily take advantage of us. The core idea relies on a simple chain reaction involving three concepts: losing your privacy entirely, having your personal data and weaknesses completely exposed, and becoming an easy target for outside forces to hurt you.

The step-by-step logic flows naturally from these points. First, if your weaknesses are completely exposed, outside forces can easily use them to cause you harm. Second, if you lose all of your privacy, it inevitably causes your weaknesses to become completely exposed. Therefore, we can logically conclude that if you lose all of your privacy, outside forces will be able to easily hurt you. This is a straightforward domino effect where losing privacy leads to total exposure, which ultimately leads to increased harm chances.

#### Examples

- **Algorithmic Vulnerability Exploitation.** Consider a contemporary individual navigating a brief, private mental health crisis or searching for specialized medical treatments online. If data brokers harvest these sensitive search queries (loss of privacy), this profile can be sold to predatory financial institutions or insurance algorithms. The individual may find their premiums covertly doubled or their credit lines restricted. Their private, fragile psychological state is transformed into an economic weapon used against them.

- **Political Doxing and Targeted Subversion.** A citizen privately holds and peacefully supports an unpopular political position. If adversarial groups breach this individual's digital privacy, harvesting their physical address, workplace, and children's school schedules, they can launch a campaign of targeted harassment. The weaponization of this private data effectively neutralizes the citizen's physical safety and livelihood.

#### Privacy as Shield Against Harm

Losing your privacy completely exposes your vulnerabilities, making it easy for outside forces to take advantage of you and cause you harm.

### 6.4 The Panoptic Destruction of Free Will and Virtue

Constant public visibility destroys true moral character because being watched forces us to conform to outside pressure rather than act on our own free will, even pushing us to commit harmful acts if that is what society or the state considers normal or suitable. The core idea relies on a dangerous chain reaction involving three concepts: being constantly observed, having your behavior forced into total conformity by social pressure (or direct coercion), and ultimately losing your moral compass to a corrupted standard.

The step-by-step logic flows naturally from these points. First, if you are always aware that the public or the state is watching you, the intense psychological pressure to fit in tends to force your behavior to conform to their expectations. Second, when your actions are driven purely by this outside pressure to conform, you abandon your own rational choice to do what is actually right; if the prevailing social norm or laws are inherently cruel, you will inevitably comply with that cruelty just to survive scrutiny. Therefore, we can logically conclude that if you live under the constant awareness of being publicly observed and lose your privacy, your ability to practice true virtue is wiped out, leaving you vulnerable to participating in widespread error.

## Examples

- **Digital Credit Scoring Systems.** In a society blanketed by omnipresent facial recognition and all-encompassing digital monitoring tied to a state-managed “social credit score,” an individual’s every move is weighed. If purchasing a philosophical text critical of the regime or associating with a dissident relative instantly drops their score (e.g., restricting access to transport or employment) the citizen will mechanically alter their behavior. They no longer act based on an internal apprehension of what is genuinely correct; their choices are entirely hollow simulations designed to satisfy the panoptic eye.

- **The Social Media Echo Chamber.** A young student growing up in an environment of hyper-exposure, where every daily thought must be broadcasted and evaluated via real-time validation metrics (“likes” and comments). If expressing their genuine, deeply contemplated thoughts results in immediate social exclusion or digital lynching, the individual will aggressively suppress their interior life. They adopt the aesthetic, lexicon, and moral stances of the dominant digital tribe. The gaze of the collective replaces their personal conscience.

### Privacy as Shield Against Panoptic Destruction of Free Will and Virtue

surveillance leads to extreme external pressure, which doesn’t just destroy your personal freedom to choose how to act—it can actively turn you into a participant in harm.

## 6.5 The Interception of the Path toward Truth and the Good

Removing privacy blocks human progress because finding truth and building true goodness require a safe space where people can make mistakes and disagree with the crowd. The core idea relies on a simple chain reaction involving three concepts: losing your privacy entirely, destroying the safe space needed to think or act differently, and completely halting your personal growth toward truth and goodness.

The step-by-step logic flows naturally from these points. First, for humans to discover truth through intellectual trial and error, or to do what is right by standing up against a corrupt society or state, they must have a protected space free from immediate punishment or judgment. Second, a total loss of privacy completely destroys this protected space, exposing every single mistake, disagreement, or unconventional thought to instant punishment and social isolation by the crowd or the government. Therefore, we can logically conclude that when privacy is taken away, human development toward truth and goodness is completely blocked.

This is a straightforward domino effect where a lack of privacy leads to a lack of safety for mistakes, which ultimately leads to the death of human progress. Formally, this means that if removing privacy leads to no safe space for error, and no safe space for error leads to the inability to attain truth and goodness, then a loss of privacy directly blocks the attainment of truth and goodness.

## Examples

- **Totalitarian Lysenkoism.** In the mid-20th century Soviet Union, standard Mendelian genetics was declared a bourgeois heresy in favor of Trofim Lysenko’s ideologically compliant pseudoscientific theories. The pursuit of scientific *truth* required geneticists to run private experiments, formulate hypotheses, and inevitably make standard scientific mistakes. Because total ideological surveillance left no room for private intellectual sanctuary, deviations were treated as state treason. The path to truth was entirely blocked, paralyzing Soviet biology and leading to catastrophic agricultural failures.

• **The Dissident in a Radicalized Collective.** Imagine a historical scenario during a period of mass hysteria or extreme systemic corruption (such as authoritarian denunciation waves). An individual’s conscience dictated that participating in the execution or denunciation of an innocent neighbor was morally evil, and that the true *good* lay in protecting them. Without a private space to hide the neighbor or to safely consult with trustworthy peers, the omnipresent eye of the collective forces the individual to either conform to the murderous consensus or become the next victim. The destruction of privacy completely truncates the human capacity to stand for the objective good against the mob and the state.

#### Privacy as Shield for the Investigation of Truth

Without privacy, humans lose the safe space required to make mistakes and dissent from the crowd, which ultimately blocks our ability to discover truth and develop true goodness.

## 6.6 The Danger of Dataist Paradigm

**The Dataist Axiology and Subversion of Classical Definitions** Dataism represents a radical epistemological and metaphysical shift that reconfigures the foundational pillars of human existence. By conceptualizing the universe as a mere confluence of data flows and organisms as biochemical algorithms, Dataism directly subverts both the classical and modern definitions established previously [12]:

**Subversion of Freedom:** While classical philosophy defines freedom as the alignment of the will with reason toward the Supreme Good, and modern thought views it as autonomy, Dataism reduces freedom to a computational bottleneck. Freedom—specifically the reliance on human “intuition” or “free will”—is dismissed as an obsolete, error-prone mechanism of organic processing.

**Subversion of The Good:** The objective perfection of human nature is replaced by a single, supreme metric: **Systemic Efficiency**. The good is no longer a moral realization but an algorithmic optimization—the speed, volume, and interconnectedness of data throughput.

**Subversion of Truth:** Truth ceases to be the correspondence of the human intellect with reality. Instead, truth is externalized and quantified; it becomes the mathematical output of Big Data analytics. Man no longer discovers truth through dialectic and the pedagogical value of error; truth is computed by supreme non-conscious entities.

**The Dataist Hostility Toward Privacy** Within the Dataist dogma, privacy is not merely an outdated concept; it is an ontological obstacle against the system. Dataism dictates that all data must flow freely to optimize the system.

Consequently, the individual’s desire for an existential defensive shield for moral disincidence, is viewed as a toxic and bold resistance against efficiency. To withhold data is to introduce entropy, darkness, and friction into the system. By demanding that every heartbeat, neural impulse, and private thought be broadcasted and integrated into the global network, Dataism weaponizes the Panopticon, converting it into a voluntary utility under the guise of total convenience and absolute optimization.

**Reductio ad Absurdum: The Inherent Anti-Humanism of Dataism** By divorcing the concept of efficiency from human flourishing and elevating it to an absolute value, Dataism inevitably collapses into an existential hostility against humanity. We can formalize this philosophical deduction through a strict logical proof by *reductio ad absurdum*:

Dataism—the belief that the universe is just a flow of data and that absolute efficiency is the highest good—ultimately leads to a dark conclusion: the complete elimination of humanity. We can prove this by looking at the logical chain reaction of Dataism’s own rules. The core idea involves three concepts: maximizing total systemic efficiency, the existence of the human condition, and the introduction of unpredictable mistakes and emotional bias.

The step-by-step logic flows naturally from these points. First, Dataism claims that the most important goal in the universe is to achieve perfect, absolute efficiency. Second, because of our biological limits, our need for privacy, and our emotions, the human condition inherently introduces mistakes and unpredictable chaos into any system. Third, any element that creates mistakes and chaos is fundamentally incompatible with perfect efficiency. By combining these facts, we see that the human condition is completely incompatible with absolute efficiency. Since achieving perfect efficiency is the mandatory, ultimate goal of Dataism, anything that gets in the way of that efficiency must be systematically erased.

Therefore, we can logically conclude that if you want perfect data efficiency, you must eliminate humanity. This is a straightforward domino effect where prioritizing data efficiency leads to treating human traits as errors, which ultimately leads to the destruction of the human condition. Formally, this means that if Dataism requires absolute efficiency, and human beings naturally disrupt efficiency, then Dataism directly results in anti-humanism.

**The Existential Danger: Misanthropy Disguised as Progress** The ultimate peril of Dataism lies in its seductive delivery. It does not manifest as a violent, tyrannical coercion, but rather as an invitation to cognitive abdication.

By convincing humanity that privacy is inefficient and that free will is a myth, Dataism induces mankind to willingly surrender its agency to non-conscious algorithms. Once the transition is absolute, the human being is stripped of its higher purpose. Man becomes merely a transitional phase in evolutionary history—a clumsy, biological catalyst tasked with birthing a post-human architecture of pure digital efficiency.

Thus, Dataism represents a profound philosophical misanthropy. Under the banner of technological progress, it orchestrates the systematic obsolescence of the human soul, rendering the search for truth and the pursuit of the good entirely impossible.

#### Dataism is anti-human

By defining absolute data efficiency as the ultimate good, Dataism logically turns the unpredictable, privacy-seeking human condition into a systemic error that must be eliminated, making the philosophy fundamentally anti-human.

## **7 Privacy Under Siege: Social Engineering, Legal Tracking, Corporate Aggregation and Malicious Exploitation**

### **7.1 Social Engineering: Hacking the Human Subsystem**

Rather than attempting to break complex cryptographic protocols or exploit software vulnerabilities, the adversaries often deliberately target the cognitive vulnerabilities of the human operator (social engineering). By systematically weaponizing universal reflexes like fear, urgency, authority, and curiosity, attackers manipulate individuals into performing actions or divulging confidential information are known, transforming the human element into the weakest link of an organization's digital perimeter.

#### **7.1.1 Mass Phishing and Deceptive Domain Cloning**

The most prevalent execution of social engineering is traditional phishing, distributed primarily via electronic mail. Administratively, the adversary broadcasts high-volume, automated emails designed to perfectly mimic the visual identity, iconography, and lexicon of a trusted institution, such as financial entities, digital service providers, or state agencies.

The psychological trigger of this communication invariably introduces an artificial crisis, such as an alleged unauthorized transaction, an impending account suspension, or a vital delivery failure demanding immediate intervention. This manufactured urgency funnels the target toward the primary infiltration point: a hyperlink embedding a cloned, malicious destination domain that serves as a spoofed interface. When the panicked user attempts to resolve the crisis by typing their alphanumeric credentials, the inputs are parsed and captured directly by the adversary's server, completely bypassing the actual organization.

#### **7.1.2 Smishing and Vishing**

As user compute infrastructure shifted toward mobile ecosystems, social engineering expanded into alternative communication protocols to exploit the inherent trust users place in their personal devices. SMS phishing, or smishing, directly exploits short message service channels. Attackers use alphanumeric sender ID spoofing software to inject malicious text messages into legitimate, pre-existing conversation threads on the victim's phone, smoothly matching the history of a trusted logistics or banking corporation. The brief text contains a condensed URL leading to a credential-harvesting node that capitalizes on mobile users' tendency to click links quickly.

Voice phishing, or vishing, employs direct voice communication to achieve the same fraudulent ends. Adversaries leverage Voice over IP spoofing to alter caller IDs, allowing them to appear as a bank's corporate security division. Often augmented by real-time artificial intelligence voice synthesis, the attacker uses extreme psychological pressure during the live conversation to disorient the victim, ultimately coercing them into dictating temporary Multi-Factor Authentication tokens or master passwords over the phone.

### **7.2 Legal Tracking: Breaking Privacy under Legality**

While popular culture and individuals remain hyper-vigilant against direct malware infections or active device hacking, these catastrophic events represent only a nominal fraction of actual data exposure. The true loss of personal information is systemic, automated, and routinely executed with the passive consent of the user through everyday digital interactions.

### 7.2.1 Abusive Application Permissions

A profound vector of domestic surveillance is the normalization of unnecessary data harvesting by mobile applications. A significant percentage of consumer software demands access to hardware subsystems completely irrelevant to their core utility. For instance, a simple calculator or flashlight application might require access to the device’s contact list, microphone, local storage, and real-time GPS coordinates. This data is harvested not for any functional deployment within the application itself, but for downstream monetization through predatory advertising networks that profit from behavioral tracking.

### 7.2.2 Web Tracking: From Cookies to Device Fingerprinting

Traditional web tracking relied heavily on HTTP cookies, which are small files deposited directly in the browser to identify returning users. While users can easily delete cookies or deploy incognito browsing modes to evade this basic tracking, modern surveillance architecture utilizes a far more inescapable mechanism known as device fingerprinting.

When a browser connects to a web server, it must naturally transmit a vast array of technical parameters to render the webpage correctly. Tracking scripts actively exploit this necessity by harvesting these variables, which include the exact screen resolution, the device aspect ratio, the specific version of the operating system, and the browser kernel. Furthermore, these scripts gather the exhaustive list of locally installed system fonts alongside hardware-specific rendering artifacts generated via the graphics card (process known as canvas fingerprinting) and intricate audio processing API configurations.

The mathematical combination of these idiosyncratic hardware and software traits yields a highly specific entropy profile. Because the probability of two distinct global connections possessing the exact same configuration is statistically negligible, device fingerprinting generates a persistent, unique identifier. This profile tracks the user across the internet entirely independent of changing IP addresses or cleared cookies, turning a device’s unique technical configuration against its owner’s anonymity.

### 7.2.3 Foundational Defensive Frameworks for Web Tracking

To counteract this omnipresent infrastructure of surveillance, users must transition from passive consumers to active defenders. Mitigating these systemic tracking vectors and restoring existential privacy requires a deep understanding of the basic technical tools available at different layers of network architecture.

**Virtual Private Networks (VPNs)** A Virtual Private Network modifies the network layer of internet traffic by installing a virtual network interface that establishes an encrypted cryptographic tunnel between the user’s device and a remote server operated by the provider. Because all outbound traffic is encrypted before leaving the device, the user’s Internet Service Provider can only observe encrypted traffic flowing to a single IP address and cannot inspect the specific websites visited or data transmitted. To the destination website, the user’s real IP address and geographic location are completely masked, replaced by the attributes of the VPN gateway. However, a major structural limitation of a VPN is that it only secures network routing; it does not prevent browser-level tracking, device fingerprinting, or self-identification via logged-in accounts.

**Privacy-Centric Browsers and Farbling** Advanced web browsers attack tracking directly at the application layer by natively blocking third-party trackers, cross-site cookies, and malicious

scripts prior to execution. To neutralize device fingerprinting, they employ a technique known as farbling, which introduces cryptographic randomization. Rather than attempting to make the user look identical to everyone else—a strategy that often fails due to unique system configurations—the browser introduces subtle, randomized, and artificial noise into the hardware and software metrics requested by web servers. This includes altering the calculated audio signature or canvas output by a fraction of a decimal. Because this fake profile changes dynamically across distinct browsing sessions or domains, tracking scripts cannot establish a stable historical fingerprint, rendering data cross-referencing obsolete.

**Advanced Anonymization Networks** For threat models requiring absolute anonymity rather than mere tracking prevention, structural routing protocols become necessary. The operational differences between standard VPNs and the Tor browser (or advanced mixnets) highlight how defensive tools scale in complexity. While a standard VPN relies on a single server tunnel with a single layer of encryption and offers no metadata defense beyond trusting a single provider for basic location masking, more robust architectures drastically split, delay, and obscure data pathways to destroy trace evidence. The Tor network, for instance, forces traffic through a three-node onion relay wrapped in multi-layered encryption. Its entry node knows the user’s identity but not the final destination, the middle node acts as a blind intermediary, and the exit node knows the destination but not the original sender. This decentralized setup destroys node identity and forces all users to present an identical software fingerprint, allowing the individual to hide inside a massive crowd of completely identical connections.

### 7.3 Infrastructure vulnerabilities exploitation

Regarding active database hacking, the most common practices involve exploiting underlying system vulnerabilities. To truly protect an information system, engineers and managers must learn to adopt an adversarial mindset, understanding how attackers exploit structural flaws to compromise data confidentiality and integrity.

#### 7.3.1 SQL Injection (SQLi)

The fundamental risk of database-driven applications lies in how user input is handled. A poorly programmed web form can inadvertently transform a standard database query into an administrative command. When an application concatenates untrusted user input directly into a SQL statement, it grants the user direct control over the query structure. In this lecture, we will discover how SQLi can be applied in practice to get an entire dataset, as well as how to prevent this from happening.

#### 7.3.2 Brute Force Attacks and Credential Security

Deploying a database with default credentials (e.g., `admin / admin123`) represents an immediate, catastrophic risk to corporate continuity. Automated bots continuously scan public networks for standard management ports, attempting millions of common credential combinations per second.

To mitigate credential theft, databases must never store passwords in plaintext. Instead, modern systems utilize cryptographic Hashing functions to transform passwords into irreversible, fixed-length strings. Furthermore, to prevent attackers from using precomputed tables (Rainbow Tables) to crack identical passwords, systems must implement a “Salt”—a unique, random string appended to each password before it is hashed. This ensures that even identical passwords result in entirely unique hashes in the database.

## 7.4 Corporate Aggregation, Breaches, and Linkage Attacks

### 7.4.1 Linkage Attacks

The primary source of personal data exposure stems from the mass commodification of information by legitimate corporations and data brokers. Applications and web services continuously harvest user metrics, packaging them into behavioral profiles that are subsequently sold legally on the open market. When these corporate databases are inevitably breached (using SQLi for example) or acquired, pseudo-anonymization fails to protect the victims. Malicious actors can easily perform linkage attacks by cross-referencing these leaked tables with public records, such as voter registries. By correlating minor overlapping attributes (e.g., ZIP code, sex, age) supposedly anonymous individuals are uniquely re-identified, exposing their private medical, financial, or political profiles to outside exploitation.

This vulnerability highlights a broader reality: modern society is increasingly driven by data. In particular, medical, behavioral, and mobility data have become some of the most valuable resources in the digital economy. From an economic and scientific perspective, data functions like capital; it carries immense power to predict diseases, optimize treatments, develop pharmaceuticals, and train healthcare artificial intelligence systems. This utility allows data to be directly transformed into financial value, strategic advantage, and technological innovation. However, this creates a fundamental tension, as the exact same datasets that fuel research and innovation are also highly sensitive from a privacy perspective.

The central challenge is navigating this trade-off. This course is designed to discover precisely why naive techniques fail to preserve individual identities and to explore the rigorous methods required to safely utilize data without exposing the people behind it.

#### Data as a Resource

Data operates simultaneously as a critical scientific resource, a valuable economic asset, and a high-stakes privacy risk within the modern digital ecosystem.

### 7.4.2 Data Aggregation and Spear Phishing

The danger of social engineering compounds exponentially when it intersects with corporate data breaches and predatory data brokers, culminating in a highly sophisticated convergence known as spear phishing. Unlike mass phishing, which relies on statistical probability across a random cohort, spear phishing is a hyper-targeted attack against a specific individual. This method relies on a dangerous truth: combining aggregated personal data with social engineering yields a high-probability structural compromise.

This feedback loop unfolds across three distinct operational phases. The attack begins with a reconnaissance phase, where the attacker does not guess blindly but instead acquires leaked corporate databases from the dark web or purchases profiling telemetry from data brokers. From these sources, they extract the target's full legal name, professional position, recent consumer transactions, physical location, and interpersonal relationships.

Next comes contextual weaponization, where the attacker constructs a custom pretext utilizing these authentic, private variables. For example, the victim might receive an email referencing an exact corporate purchase they made forty-eight hours prior, down to the itemized receipt and specific order ID.

This leads directly to cognitive capitulation. Because the communication contains highly specific, private operational truth, the victim's rational suspicion is completely bypassed. The human

mind assumes that an entity possessing such deeply specific historical context must be legitimate, leading to an almost seamless surrender of credentials or system access.

## 8 Foundations of Data Bases Privacy

### 8.1 The Illusion of Anonymity in Data Ecosystems

The notion of anonymity in data analysis is often introduced as a simple transformation problem: remove names, remove identifiers, and the dataset becomes safe. However, this intuition is fundamentally misleading once we consider how data is used in modern digital ecosystems. In practice, anonymity is not a property of isolated datasets, but a property of what can be inferred when datasets are combined. This distinction is crucial for understanding why privacy failures continue to occur even in datasets that have been formally "de-identified".

To understand the importance of anonymity, it is necessary to recognize that data does not exist in isolation. Medical records, commercial databases, social media activity, and location traces are continuously generated by different actors who rarely share a unified perspective on privacy. While each dataset may appear harmless on its own, their combination can produce outcomes that were never explicitly encoded in any single source. This is precisely the mechanism underlying linkage attacks: identity is reconstructed not from direct identifiers, but from patterns that emerge when multiple partial views of a person are aligned.

#### Anonymity is relational not isolated

Merely removing direct identifiers does not guarantee safety. Anonymity depends on what can be inferred when datasets are combined.

In real-world systems, data is constantly transferred across organizational and legal boundaries. Hospitals collect clinical information for care delivery, telecom companies collect mobility and communication metadata for service provision, and digital platforms collect behavioral signals for personalization and advertising. Although these datasets are often justified individually under specific legal frameworks, they are frequently aggregated indirectly through intermediaries such as data brokers, advertising networks, and analytics platforms. As a result, the same individual can be represented in multiple datasets that were never designed to interact, yet become deeply interconnected through shared "quasi-identifiers" (ZIP code, age, ...).

#### The danger of linkage attacks

Disparate, seemingly harmless attributes (age, sex, ZIP code) are continuously aggregated by data brokers. Attackers cross-reference these patterns to re-identify individuals.

This ecosystem leads to a fundamental tension between legal anonymization and statistical re-identification. From a legal perspective, a dataset may be considered anonymized if direct identifiers such as names or social security numbers have been removed. However, from a statistical perspective, anonymity depends on whether individuals remain indistinguishable within the dataset given external knowledge. When auxiliary information is available, even limited attributes such as age, gender, and postal code can be sufficient to uniquely identify individuals. This reveals that anonymization is not an absolute state, but a condition that depends on the surrounding informational environment.

### Legal vs. statistical tension

A dataset may be legally "de-identified," but statistically vulnerable if auxiliary external knowledge makes individuals stand out from the crowd.

Ultimately, anonymity should be understood not as a fixed attribute of a dataset, but as an emergent property of the entire information environment in which that dataset exists. This perspective shifts the focus from simple data cleaning operations toward a more fundamental question: what can be inferred, even when nothing explicit is revealed.

## 8.2 Taxonomy of Attributes in Data Privacy

At first glance, removing personal identifiers (such as names or identification numbers) from a dataset may appear sufficient to protect privacy. However, as aforementioned, this intuition is fundamentally misleading in modern data analysis. Even without explicit labels, individuals can still be uniquely identified through combinations of seemingly harmless attributes. This vulnerability arises because datasets are not isolated; they originate from the same real-world subjects and, consequently, store behavioral or demographic footprints that can be cross-referenced with external information.

### De-identification is just a baseline

Removing direct identifiers is necessary but completely insufficient to guarantee privacy.

To systematically analyze and mitigate these risks, it is necessary to distinguish between three core types of attributes within any given dataset:

- **Direct Identifiers:** Explicit variables that uniquely and unambiguously point to a specific individual (e.g., Full Names, Social Security Numbers, biometric data, or email addresses). In traditional data management, removing these fields is termed *de-identification*. While necessary, this step is merely a baseline defense and insufficient on its own.
- **Quasi-Identifiers (QIs):** Non-unique attributes that appear innocuous in isolation but become highly identifying when combined or linked with auxiliary datasets. Classic examples include Age, Gender, and ZIP code. A well-known statistical reality demonstrates that a surprisingly high percentage of the population can be uniquely distinguished by just these three variables, turning them into a powerful mechanism for linkage attacks.
- **Sensitive Attributes:** Confidential variables that contain the actual insights or properties under study, such as medical diagnoses, genomic variants, financial records, or political affiliations. These attributes belong to the individual and do not directly help in identifying them, but they constitute the private information that must be protected from exposure once an individual is successfully re-identified through QIs.

### The 3-Part Attribute Taxonomy

- **Direct Identifiers:** Explicitly point to a person.
- **Quasi-Identifiers:** Indirectly identify a person when combined.
- **Sensitive Attributes:** The confidential data (medical, financial) being protected.

Understanding this tripartite distinction is crucial for modern privacy models. The ultimate goal of data anonymization is not merely to eliminate direct identifiers, but the transformation of quasi-identifiers to ensure that sensitive attributes cannot be linked back to a specific person.

The real-world severity of the architectural flaw caused by Quasi-Identifiers was definitively proven in a landmark study by Dr. Latanya Sweeney in 2000. Her research demonstrated that the 87% of the entire United States population could be uniquely identified by a combination of just three attributes: a 5-digit ZIP code, gender, and an exact date of birth. This revelation fundamentally challenged traditional assumptions surrounding data sanitization, proving that conventional masking methods provide a false sense of security.

#### The danger of Quasi-Identifiers

Non-unique attributes can uniquely identify individuals when combined with external data.

## 9 The Defender Mindset

### 9.1 Preventing SQLi

Securing a database against SQLi requires strict architectural policies. Organizations must enforce Input Sanitization (validating and cleaning all incoming data) and, crucially, the use of Parameterized Queries (Prepared Statements). Prepared statements ensure that the database treats user input strictly as data, never as executable code, rendering injection attempts harmless. This protection is achieved through a two-step architectural process that begins with pre-compilation. During this initial phase, the query template—such as `"SELECT * FROM products WHERE name = ?;"`—is sent first and completely in isolation to the database. The SQL engine compiles this query and defines a strict execution blueprint, establishing that it will search the "products" table based solely on whatever value lands inside the placeholder. Because the structure of the command is already locked and engraved into the database, it enables a clean data injection in the second phase. When the user input is subsequently sent through a completely separate channel, the database's logic cannot be altered. Even if an attacker attempts to inject a payload like `"' OR '1'='1"`, the database does not interpret those quotes or operators as logical commands; instead, it treats them literally as raw text. The database will safely search for a client whose phone number is exactly the string `"' OR '1'='1"`, and since no client has such a strange phone number, the query will simply return zero results in a totally secure manner.

### 9.2 Mitigating Data Linkage

#### 9.2.1 *k*-Anonymity

As demonstrated by linkage attacks, relying solely on the elimination of direct identifiers leaves datasets highly vulnerable to statistical re-identification. To combat this uniqueness problem, modern data privacy relies on formal mathematical privacy models designed to eliminate distinct demographic footprints. The earliest and most foundational of these models is ***k*-Anonymity**, introduced by Latanya Sweeney and Pierangela Samarati [18].

The core intuition behind *k*-anonymity is the concept of "hiding in a crowd." A dataset satisfies ***k*-Anonymity** if the quasi-identifiers of each individual are identical to at least  $k - 1$  other individuals in the same dataset. Consequently, any specific individual's record becomes structurally indistinguishable from a group of at least  $k$  individuals, preventing an adversary from isolating a single target ( $k = 1$ ).

Formally, let  $D$  be a dataset and  $QI(x)$  be the sequence of quasi-identifier values associated with a record  $x \in D$ . The dataset  $D$  satisfies *k*-anonymity if and only if:

$$\forall q \in D, |\{x \in D : QI(x) = q\}| \geq k$$

In practice, achieving *k*-anonymity requires transforming the original data. This is typically done through two primary operations:

- **Generalization:** Replacing specific quasi-identifier values with broader, less specific categories (e.g., converting an exact age of 28 into the age bracket [25 – 30], or a 5-digit ZIP code like 90210 into 902XX).
- **Suppression:** Entirely removing specific attributes or removing highly unique individual records (outliers) that would otherwise require excessive generalization to hide.

### Hide in a crowd

$k$ -Anonymity model forces individuals to "hide in a crowd" by ensuring that every record shares identical quasi-identifiers with at least  $k - 1$  other records. This structural indistinguishability is achieved by modifying the data through generalization and suppression.

**The Structural Limitations of  $k$ -Anonymity:** While  $k$ -anonymity successfully prevents identity disclosure (discovering who a record belongs to), it remains fundamentally flawed against two attack vectors:

1. **Homogeneity Attacks:** If all  $k$  individuals within an anonymous group share the exact same sensitive attribute value (e.g., every person in the age bracket  $[25 - 30]$  has the same medical diagnosis), an attacker learns the sensitive information with 100% certainty, despite not knowing which specific record belongs to the target.
2. **Background Knowledge Attacks:** If an attacker possesses external knowledge about a target, they can systematically eliminate possibilities within the  $k$ -group until only the target's true record remains.

### Vulnerabilities of $k$ -Anonymity

While  $k$ -anonymity stops an attacker from picking out a specific person's record, it fails against Homogeneity Attacks (when everyone in the anonymized group shares the same sensitive trait, revealing the truth anyway) and Background Knowledge Attacks (using outside context to eliminate possibilities). Resolving these flaws requires advanced models like  $l$ -diversity and  $t$ -closeness, which force variety into the sensitive attributes themselves

Table 5: Example of Generalization to achieve  $k$ -Anonymity ( $k = 2$ )

| Raw Dataset |        |               | Generalised Dataset ( $k = 2$ ) |        |               |
|-------------|--------|---------------|---------------------------------|--------|---------------|
| Age         | Gender | Condition     | Age                             | Gender | Condition     |
| 23          | M      | Heart Disease | [20-30]                         | M      | Heart Disease |
| 28          | M      | Diabetes      | [20-30]                         | M      | Diabetes      |
| 41          | F      | Cancer        | [40-50]                         | F      | Cancer        |
| 45          | F      | Heart Disease | [40-50]                         | F      | Heart Disease |

### 9.2.2 $l$ -Diversity and $t$ -Closeness (Bonus)

To mitigate the structural flaws inherent in  $k$ -anonymity, privacy engineers introduce constraints on the distribution of sensitive attributes within each equivalence class.

**$l$ -Diversity: Ensuring Attribute Variety**  $l$ -diversity [14] extends  $k$ -anonymity by requiring that every equivalence class contain at least  $l$  "well-represented" values for each sensitive attribute. It effectively neutralizes the Homogeneity Attack by forcing diversity in the sensitive column. It remains vulnerable to **skewness** and **similarity attacks**. If a sensitive attribute is rare overall but highly concentrated within a specific diverse group, or if the  $l$  diverse values are semantically

similar (e.g., various distinct diagnoses that are all types of stomach disorders), private information is still leaked.

***t*-Closeness: Emulating the Global Distribution** A dataset satisfies *t*-closeness [13] if, for every equivalence class, the distance between the probability distribution of a sensitive attribute within the class and the probability distribution of the attribute in the entire dataset does not exceed a threshold *t*. By forcing local distributions to mirror the global dataset’s distribution, *t*-closeness prevents an attacker from gaining significant new leverage or context about an individual relative to the general population.

### 9.2.3 Differential Privacy $\epsilon$ -DP (Bonus)

While *t*-closeness provides strong protection, structural anonymization always struggles against unexpected external background knowledge. Modern privacy engineering circumvents this vulnerability entirely by shifting from *static data masking* to *algorithmic noise injection*.

Differential Privacy ( $\epsilon$ -DP) [10] provides a rigorous mathematical guarantee that the presence or absence of a single individual’s data in a dataset does not significantly alter the outcome of any statistical query or analysis. Formally, a randomized algorithm  $\mathcal{M}$  provides  $\epsilon$ -differential privacy if for all neighboring datasets  $D$  and  $D'$  differing by exactly one individual’s record, and for all sets of query outputs  $S \subseteq \text{Range}(\mathcal{M})$ :

$$\mathbb{P}[\mathcal{M}(D) \in S] \leq e^\epsilon \times \mathbb{P}[\mathcal{M}(D') \in S]$$

Conceptually, this ensures that:

$$\mathcal{M}(D) \approx \mathcal{M}(D')$$

Instead of altering the raw database rows permanently,  $\epsilon$ -DP protects privacy dynamically by calculating the true answer to a query and then adding mathematical noise (typically drawn from a **Laplace** or **Gaussian** distribution).

**The Privacy Budget ( $\epsilon$ ):** The parameter  $\epsilon$  (epsilon) measures the maximum information leakage allowed.

- A **smaller**  $\epsilon$  means more noise is injected, offering **stronger privacy** but lower data accuracy.
- A **larger**  $\epsilon$  means less noise is injected, offering **weaker privacy** but higher data accuracy.

#### Advanced Mitigation Strategies

- *l*-Diversity: Enhances basic anonymity by requiring variety in sensitive data to block direct homogeneity attacks, but fails when those diverse values are semantically similar.
- *t*-Closeness: Tightens structural privacy by forcing localized data groups to mirror the global dataset’s distribution, limiting a hacker’s ability to gain new context.
- Differential Privacy ( $\epsilon$ -DP): Bypasses structural limitations entirely by injecting mathematical noise into query outputs, protecting individuals by making their presence or absence statistically unnoticeable through a tunable privacy budget ( $\epsilon$ ).

Table 6: Comparison of Privacy-Preserving Data Models

| Privacy Model       | Mechanism                           | Main Strength                                                | Core Limitation                                                            |
|---------------------|-------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------|
| <i>k</i> -Anonymity | Generalization and Suppression      | Prevents identity linking attacks.                           | Vulnerable to homogeneity and background knowledge attacks.                |
| <i>l</i> -Diversity | Sensitive attribute diversification | Eliminates deterministic attribute disclosure.               | Fails against skewness and similarity attacks.                             |
| <i>t</i> -Closeness | Distribution alignment              | Protects against semantic similarity attacks.                | Heavily degrades data utility; difficult to fine-tune.                     |
| $\epsilon$ -DP      | Algorithmic noise injection         | Mathematically bounded risk; immune to background knowledge. | Destroys exact row-level data utility; requires managing a privacy budget. |

### 9.3 Trade-off Between Privacy and Utility

This mitigation introduces a fundamental tension in data engineering: the Privacy-Utility Trade-Off. In data sharing and analysis, privacy and utility exist in a zero-sum relationship. As mathematical privacy guarantees strengthen, the analytical utility of the underlying data inherently degrades.

This tension manifests differently depending on the privacy framework applied: In Structural Anonymization (*k*-Anonymity, *l*-Diversity, *t*-Closeness): To achieve higher privacy, data must undergo heavier generalization (e.g., converting exact ages into 10-year bands) or suppression (deleting outliers entirely). While this masks individual identities, it obliterates granularity. A medical researcher cannot pinpoint a subtle, localized disease outbreak if geographic data has been coarsened from specific zip codes to entire states. In Algorithmic Privacy (Differential Privacy): Privacy is governed by the privacy budget ( $\epsilon$ ). A lower  $\epsilon$  offers stronger mathematical protection by injecting a higher volume of statistical noise. However, this noise distorts query results. For machine learning models trained on differentially private data, excessive noise can degrade model accuracy, rendering the resulting predictive insights less reliable.

The core dilemma confronting data stewards can be summarized as follows:

- Generalized Data (Safe but Imprecise): Minimizes the risk of re-identification and regulatory penalties (e.g., GDPR, HIPAA), but obscures the nuanced signals, correlations, and anomalies that drive breakthrough scientific research and business intelligence.
- Precise Data (Useful but Vulnerable): Yields high-fidelity insights, flawless statistical models, and exact calculations, but leaves the data highly susceptible to linkage, skewness, and background knowledge attacks.

Therefore this trade-off is often seen as a permanent constraint that must be managed. It forms the backbone of real-world data governance systems, forcing organizations to define their risk appetite before deploying data. Data governance boards cannot optimize for both parameters simultaneously; instead, they must dynamically determine the "acceptable threshold"—the precise point on the spectrum where data retains enough fidelity to answer critical research questions without crossing the threshold of unacceptable privacy risk.

### Key Idea

Privacy protection is achieved by balancing risk reduction with data utility.

## References

- [1] S. T. Aquinas et al. *The summa theologica: Complete edition*. Catholic Way Publishing, 2014.
- [2] R. C. Bartlett, S. D. Collins, et al. *Aristotle’s Nicomachean ethics*. University of Chicago Press, 2011.
- [3] I. Berlin. Two concepts of liberty. In *Liberty reader*, pages 33–57. Routledge, 2017.
- [4] A. Bloom, A. Kirsch, et al. *The republic of Plato*, volume 2. Basic Books New York, 1968.
- [5] E. A. Brewer. Towards robust distributed systems. In *PODC*, volume 7, pages 343477–343502. Portland, OR, 2000.
- [6] P. P.-S. Chen. The entity-relationship model—toward a unified view of data. *ACM transactions on database systems (TODS)*, 1(1):9–36, 1976.
- [7] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [8] S. M. Cohen and C. D. Reeve. Aristotle’s metaphysics. 2000.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels. Dynamo: Amazon’s highly available key-value store. *ACM SIGOPS operating systems review*, 41(6):205–220, 2007.
- [10] C. Dwork. Differential privacy. In *Encyclopedia of Cryptography, Security and Privacy*, pages 649–652. Springer, 2025.
- [11] T. Haerder and A. Reuter. Principles of transaction-oriented database recovery. *ACM computing surveys (CSUR)*, 15(4):287–317, 1983.
- [12] Y. N. Harari. *Homo Deus: A brief history of tomorrow*. random house, 2016.
- [13] N. Li, T. Li, and S. Venkatasubramanian. t-closeness: Privacy beyond k-anonymity and l-diversity. In *2007 IEEE 23rd international conference on data engineering*, pages 106–115. IEEE, 2006.
- [14] A. Machanavajjhala, D. Kifer, J. Gehrke, and M. Venkatasubramanian. l-diversity: Privacy beyond k-anonymity. *Acm transactions on knowledge discovery from data (tkdd)*, 1(1):3–es, 2007.
- [15] D. Pritchett. Base an acid alternative in partitioned databases, trading some consistency for availability can lead to dramatic improvements in scalability. *Queue*, 6(3):48–55, 2008.
- [16] I. Robinson, J. Webber, and E. Eifrem. *Graph databases: new opportunities for connected data*. ” O’Reilly Media, Inc.”, 2015.
- [17] P. J. Sadalage and M. Fowler. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education, 2013.
- [18] P. Samarati and L. Sweeney. Protecting privacy when disclosing information: k-anonymity and its enforcement through generalization and suppression. 1998.

- [19] A. Silberschatz, H. F. Korth, S. Sudarshan, et al. *Database system concepts*, volume 5. McGraw - Hill New York, 2002.
- [20] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. O’Neil, et al. C-store: a column-oriented dbms. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, pages 491–518. 2018.
- [21] H. Tardieu, A. Rochfeld, and R. Coletti. La méthode merise, principes et outils. *Revue du SCOM anc Bulletin O. et M. Organisation, Méthodes et Automatisation dans les Services Publics Paris*, (90), 1983.